Универзитет у Београду
Електротехнички факултет

ДИПЛОМСКИ РАД

СИСТЕМ ЗА ДИСТРИБУИРАНУ ПАКЕТНУ ОБРАДУ У ОБЛАКУ

Ментор
проф. др Милош Цветановић

Студент
Кристијан Ковачевић, 656/22

Београд, 2026

Zahvalnica

Ovaj rad predstavlja kumulativni trud porodice Kovačević, jer bez njihove podrške ne bih stigao dovde. Zahvaljujem se za sve upućene molitve, za strpljenje i razumevanje tokom godina studiranja i za bezuslovnu podršku u trenucima kada je bilo najteže.

Za svaki pređeni kilometar i za dane provedene daleko od kuće, u dalekim zemljama. Njegovo odsustvo je bilo cena koju je platio da bih ja mogao da studiram ono što volim, i pesma koju sam kao dete pevao čekajući da se vrati ostaje jedna od najtoplijih uspomena tog vremena.

Pozdravite moga tatu
tamo, iza sedam gora
pozdravite moga tatu
na talasima plavog mora

„Nema NE MOGU, NEĆU i NE UMEM.“ Danas se ispostavlja da i hoću, i mogu, a ono što ne umem - naučiću.

Posebnu zahvalnost dugujem prijateljima koji su bili uz mene za svako pitanje, nedoumicu tokom studija. Na strpljenju, savetima i tome što su uvek bili dostupni kada mi je bilo najpotrebnije.

Okolina: šta ćes upisivati?
Ja: ne znam, al znam da neću etf

Slava Bogu za sve

Sažetak

Obrada velikih skupova podataka na jednoj mašini postaje neefikasna kada veličina ulaznih podataka premaši kapacitet jednog procesa da ih obradi u prihvatljivom vremenu, što otvara potrebu za distribuiranim izvršavanjem zadataka na više paralelnih radnika.
Ovaj rad se bavi problemom orkestracije takvog distribuiranog izvršavanja: kako pouzdano podeliti ulazni skup podataka, pokrenuti odgovarajući broj izvršilaca u kontejnerizovanom okruženju, pratiti njihov status i sastaviti parcijalne rezultate u jedinstven izlaz, bez ručne intervencije operatera.

Postojeća rešenja ovog problema kao sto su Apache Hadoop MapReduce, Apache Spark, sistemi za red zadataka poput Celery-ja, servisi poput AWS Batch-a i alati za orkestraciju tokova, gde se Apache Airflow izdvaja kao primer, rešavaju sličan problem, ali svaki od njih uvodi sloj apstrakcije koji sakriva orkestraciju iza sopstvenog izvršnog modela, čime otežava direktno posmatranje i objašnjavanje ponašanja Kubernetesa.

Predloženo rešenje je platforma koja direktno koristi Kubernetes Indexed Job kao jedinicu distribuiranog izvršavanja, uz FastAPI backend koji upravlja životnim ciklusom posla i React frontend za nadzor, čime se zadržava puna providnost nad orkestracionim slojem uz minimalnu operativnu složenost pogodnu za lokalno okruženje. Uporedna analiza je sprovedena eksperimentalno, merenjem stvarnog vremena izvršavanja i paralelne efikasnosti sistema pri promeni broja worker-a (1 do 32) za skupove podataka od 100.000 i 10.000.000 redova.

Glavni numerički rezultati pokazuju da sistem za veliki skup podataka (10 miliona redova) dostiže ubrzanje od približno 2,2 puta u odnosu na jednog worker-a, sa platoom performansi izmedju 8 i 32 worker-a, dok se za mali skup podataka (100.000 redova) ubrzanje ne ostvaruje, vreme izvršavanja sa 32 worker-a je duže nego sa jednim, što potvrdjuje pretpostavku da je sistem ograničen deljenim I/O kanalom, a ne procesorskom snagom.

Sadržaj

1. Uvod 1
2. Pregled postojećih rešenja 4
    2.1. Postojeća rešenja 4
        2.1.1. Apache Hadoop MapReduce 4
        2.1.2. Apache Spark 4
        2.1.3. Sistemi zasnovani na redu zadataka (Celery) 4
        2.1.4. Upravljani cloud servisi za paketnu obradu (AWS Batch) 5
        2.1.5. Orkestracija tokova poslova (Apache Airflow i Argo Workflow) 5
    2.2. Dobre i loše strane postojecih rešenja 6
    2.3. Ciljne karakteristike rešenja 7
    2.4. Početne pretpostavke 7
    2.5. Predlog rešenja problema na globalnom nivou 8
3. Pregled detalja rešenja 10
    3.1. Kubernetes Indexed Job i API mehanizam 10
    3.2. Najbitniji elementi realizacije resenja 11
    3.3. Uslovi pod kojima se rešenje primenjuje 11
4. Implementacija rešenja 12
    4.1. Opis donetih projektnih odluka 17
    4.2. Mogućnosti za modifikaciju rešenja 18
    4.3. Korisničko uputstvo 18
        4.3.1. Primer tipičnog scenarija korišćenja 19
        4.3.2. Opis izvornog koda i tehnički detalji za izgradnju izvršnog koda 20
        4.3.3. Format ulaznih i izlaznih datoteka 20
        4.3.4. Detaljan opis instalacije i pokretanja 21
        4.3.5. Primeri za proveru izvršenog koda 22
5. Pregled dobijenih rezultata 23
    5.1. Sistematsko manuelno testiranje graničnih slučajeva 23
    5.2. Automatizovani test paket 23

5.3. Testiranje performansi pri promeni broja worker-a 25
5.3.1. Rezultati za skup podataka od 10.000.000 redova 26
5.3.2. Rezultati za skup podataka od 100.000 redova 29
5.3.3. Diskusija i provera početnih pretpostavki 31
5.3.4. Pretnje validnosti benchmark eksperimenta 32
5.3.5. Veza sa uporednom odradjenom analizom 32
6. Zakljucak 34
7. Literatura 36
8. Prilozi 37
8.1. Kompletna struktura repozitorijuma 37
8.2. Kompletna specifikacija konfiguracionih parametara 38
8.3. Kompletna specifikacija modela podataka posla 39
8.4. Primer strukture ulazne i izlazne CSV datoteke 40

1. Uvod

Poslednjih petnaestak godina obrada podataka je prešla put od jednog moćnog servera koji izvršava sekvencijalni program, preko klastera specijalizovanih mašina namenjenih velikim podacima (Hadoop, Spark), do savremenog cloud-native pristupa u kojem se svaki deo posla pakuje u kontejner, a rasporedjivanje, skaliranje i oporavak od otkaza prepuštaju zajedničkom orkestracionom sloju. Kubernetes se u tom kontekstu nametnuo kao de facto standard za orkestraciju kontejnera: obezbedjuje deklarativni model u kojem se željeno stanje sistema (broj replika, resursi, politika ponovnog pokretanja) opisuje kroz API objekte, a kontrolna petlja (control loop) samog Kubernetesa preuzima odgovornost da stvarno stanje klastera dovede u sklad sa željenim.

Za razliku od servisa dugog veka trajanja, poput veb servera ili baze podataka, obrada podataka u paketima (batch processing) ima drugačiji profil: posao ima jasan početak i kraj, sastoji se od većeg broja nezavisnih ili slabo zavisnih jedinica rada koje se mogu izvršavati paralelno, a uspeh se meri time da li su sve jedinice rada uspešno završene. Kubernetes za ovu namenu nudi poseban resurs, Job, čija je semantika prilagodjena upravo paketnom izvršavanju: Job garantuje da se odredjeni broj instanci pod-a izvrši do uspešnog završetka, uz mogućnost paralelizma i, od verzije koja podržava Indexed completion mode, uz mogućnost da se svakoj instanci dodeli jedinstveni redni broj koji ona može iskoristiti da odredi koji deo posla joj pripada. Upravo je ovaj mehanizam, Kubernetes Job sa Indexed completion mode-om, iskorišćen kao poluga na kojoj počiva rešenje opisano u ovom radu.

Paralelno sa razvojem orkestracionih platformi, razvijala se i čitava porodica specijalizovanih sistema za obradu velikih podataka, od MapReduce modela koji je popularizovao Google, preko Apache Hadoop-a i Apache Spark-a, do modernih sistema za tokove poslova (workflow) kao što je Apache Airflow. Svi ovi sistemi rešavaju problem distribuiranog izvršavanja, ali ga po pravilu rešavaju iznad Kubernetesa ili nezavisno od njega, uvodeći sopstveni izvršni model, sopstvenog raspoređivača (scheduler) i sopstvene apstrakcije. Za potrebe ovog rada, čiji je fokus na infrastrukturi, a ne na logici same obrade podataka, bilo je važno da orkestracija ostane direktno vidljiva i objašnjiva, bez dodatnog sloja koji bi sakrio kako se Job, Pod i deljeno skladište zaista ponašaju.

Problem koji se rešava u ovom radu može se formulisati na sledeći način: korisnik poseduje CSV (comma-separated values) datoteku sa proizvoljnim brojem redova i želi da nad svakim redom izvrši istu operaciju obrade, uz zahtev da
se obrada izvrši brže nego što bi to bio slučaj sa sekvencijalnim izvršavanjem na jednom procesu, korišćenjem više paralelnih izvršilaca (worker-a) koji rade nezavisno jedan od drugog. Sistem treba da omogući korisniku da:

1. otpremi CSV datoteku i odabere broj worker-a koji će učestvovati u obradi
2. automatski podeli ulazni skup podataka po redovima na onoliko delova (chunk-ova) koliko je odabrano worker-a
3. automatski kreira i upravlja Kubernetes Job i Pod resursima koji izvršavaju obradu
4. prati status izvršavanja u realnom vremenu
5. sastavi (agregira) parcijalne rezultate svih worker-a u jedan konačan izlazni fajl koji korisnik može preuzeti

6. automatski očisti sve privremene Kubernetes i generisane resurse nakon završetka posla, bez ručne intervencije.

Namerno je izabran generički zadatak obrade (a ne konkretan poslovni proces poput analize logova ili transformacije slika), jer je fokus rada na infrastrukturi, na tome kako se obrada distribuira, orkestruje, nadgleda i objedinjava.

Poseban deo problema, koji je tokom realizacije postao podjednako važan koliko i sama funkcionalna implementacija, jeste pitanje da li i pod kojim uslovima distribucija posla na više worker-a zaista donosi ubrzanje. Naivna
pretpostavka da veći broj worker-a uvek znači brže izvršavanje se pokazala netačnom već u ranoj fazi eksperimentisanja, što je otvorilo drugi, empirijski deo problema: izmeriti kako se vreme izvršavanja, ubrzanje i efikasnost sistema
menjaju sa brojem worker-a, za različite veličine ulaznog skupa podataka, i utvrditi da li je uska grla sistema procesorska snaga ili nešto drugo (u ovom slučaju, deljeni I/O kanal ka skladištu). Ovaj deo problema, iako nije eksplicitno naveden u prvobitnom predlogu teme, prirodno proizilazi iz zahteva da se sistem nadgleda i meri, i predstavlja jedan od centralnih doprinosa rada

Predloženo i implementirano rešenje je platforma sastavljena iz tri sloja:
1. React frontend za otpremanje CSV datoteke, pokretanje posla i nadzor statusa i metrika
2. FastAPI backend koji upravlja životnim ciklusom posla, od validacije ulaza, preko particionisanja i predaje Kubernetes Job-a, do provere statusa, agregacije rezultata i čišćenja
3. Kubernetes klaster (lokalno, preko minikube-a) koji izvršava sam posao kroz jedan Indexed Job sa N worker Pod-ova, gde svaki pod na osnovu sopstvenog rednog broja (JOB_COMPLETION_INDEX) zna tačno koji deo ulaznih podataka da obradi. Deljeno skladište izmedju host mašine i Kubernetes klastera realizovano je preko hostPath volumena podržanog `minikube mount` mehanizmom (9p protokol).

Sve funkcionalnosti definisane u obimu rada su implementirane i pokrivene automatizovanim testovima (82 testa) i end-to-end demonstracionim skriptom koji se izvršava nad stvarnim klasterom. Pored funkcionalne ispravnosti, sprovedeno
je i sistematsko merenje performansi kroz namensku benchmark skriptu koja menja broj worker-a (1, 2, 4, 8, 10, 12, 14, 16, 20, 24, 32) za dva skupa podataka različite veličine (100.000 i 10.000.000 redova). Rezultati potvrdjuju da je sistem, u trenutnoj konfiguraciji sa jednim deljenim hostPath/9p mostom kao skladištem, ograničen brzinom ulazno-izlaznih operacija, a ne procesorskom snagom.

Poglavlje "Pregled postojećih rešenja", analizira postojeće pristupe distribuiranoj paketnoj obradi podataka (Hadoop MapReduce, Apache Spark, Celery i sistemi zasnovani na redu poruka, upravljani cloud servisi poput AWS Batch-a, alati za orkestraciju tokova poput Apache Airflow-a i Argo Workflows-a), sa osvrtom na prednosti i nedostatke svakog rešenja iz ugla problema definisanog u ovom radu. Na osnovu te analize izvode se ciljne karakteristike koje predloženo rešenje treba da ispuni, početne pretpostavke pod kojima se rešenje razvija i predlog rešenja na globalnom, arhitekturnom nivou.

Poglavlje "Pregled detalja rešenja", opisuje postupak i tehnike korišćene prilikom realizacije, najbitnije elemente realizacije, uslove pod kojima se rešenje primenjuje, implementaciju sistema sa opisom funkcionalnosti, ključnih modula, karakterističnih delova logike i donetih projektnih odluka, kao i korisničko uputstvo koje pokriva funkcije sistema, tipične scenarije korišćenja, opis izvornog koda, format ulaznih i izlaznih datoteka, instalaciju i primere za proveru rada sistema.

Poglavlje "Pregled dobijenih rezultata", prikazuje sprovedeno testiranje rešenja. Funkcionalno testiranjei testiranje performansi pri promeni broja worker-a.

Poglavlje "Zaključak", sumira u kojoj meri su ostvareni ciljevi postavljeni u zadatku diplomskog rada, kritički sagledava najznačajnije rezultate, njihove pogodnosti, ograničenja i moguća područja primene.

Poglavlje "Literatura", navodi izvore koji su korišćeni prilikom izrade rada, a poglavlje "Prilozi", sadrži pomoćne detaljne UML dijagrame, kompletnu specifikaciju API-ja i konfiguracionih parametara.

2. Pregled postojećih rešenja

Problem distribuirane obrade podataka u paketima star je koliko i sama potreba da se obradi više podataka nego što jedna mašina može da obradi u razumnom vremenu. Kroz godine su se iskristalisale nekoliko porodica rešenja, od kojih je svaka nastala kao odgovor na ograničenja prethodne.

2.1. Postojeća rešenja

U nastavku su analizirana rešenja koja su relevantna za problem definisan u ovom radu; rešenja koja se bave orkestracijom distribuiranog izvršavanja paketnih poslova.

2.1.1. Apache Hadoop MapReduce

Hadoop MapReduce deli posao na Map i Reduce faze, pri čemu je paralelizam ugradjen u sam model: Map faza se prirodno izvršava paralelno nad particijama ulaznih podataka, a Reduce faza agregira rezultate. Interno razvijen distribuirani fajl sistem deli svaku datoteku na blokove (128 MB) koje replicira (podrazumevano tri kopije) preko čvorova klastera, a rasporedjivač (scheduler) dodeljuje kontejnere zadacima na osnovu dostupnosti CPU/RAM resursa.

2.1.2. Apache Spark

Apache Spark je nastao kao odgovor na ograničenja MapReduce modela, pre svega na to što MapReduce upisuje medjurezultate na disk izmedju faza, što ga čini sporim za iterativne algoritme i za interaktivnu analizu. Spark uvodi apstrakciju otpornih distribuiranih skupova podataka (Resilient Distributed Datasets, RDD) koji se, kad god je to moguće, drže u memoriji klastera, a izvršni plan se optimizuje pre pokretanja (lazy evaluation, DAG scheduler). Spark može da radi nad sopstvenim klaster menadžerom (Spark Standalone), nad YARN-om, ili nad Kubernetesom (Spark Kubernetes Operator/scheduler), pri čemu u poslednjem slučaju Spark sam kreira Executor pod-ove kao zaseban resurs koji Kubernetes vidi, ali čijim životnim ciklusom upravlja Spark-ov sopstveni driver proces, a ne direktno Kubernetes Job kontroler. Nakon pokretanja posla komandom `spark-submit`, koja se poziva na driver program, taj program gradi DAG transformacija nad RDD/DataFrame apstrakcijom, a Spark-ov unutrašnji scheduler (DAGScheduler, TaskScheduler) deli DAG na faze i zadatke koji se rasporedjuju na executor pod-ove.

Ovo je upravo glavni nedostatak Spark-a iz ugla problema definisanog u ovom radu: iako Spark tehnički „koristi” Kubernetes (kao target za pokretanje pod-ova), sam čin orkestracije (deljenje posla, praćenje napretka, rasporedjivanje zadataka na worker-e) odvija se unutar Spark-ovog sopstvenog driver procesa, izvan domašaja Kubernetes Job/Pod semantike koja je predmet proučavanja ovog rada.

2.1.3. Sistemi zasnovani na redu zadataka (Celery)

Celery (u Python ekosistemu) predstavlja drugačiji obrazac: zadaci se serijalizuju i stavljaju u red poruka (RabbitMQ, Redis), a fleksibilan broj worker procesa (koji mogu, ali ne moraju biti Kubernetes pod-ovi) preuzima zadatke iz reda i izvršava ih.

Paralelizam se postiže povećanjem broja worker procesa/instanci koje slušaju isti red, a rezultati se, ako su potrebni, čuvaju u posebnom „result backend-u" (npr. Redis ili baza podataka), koji klijent koji je poslao zadatak može periodično proveravati (polling) ili čekati preko posebnog mehanizma obaveštavanja.

Particionisanje jednog ulaznog fajla na N delova, predaja tih delova kao N pojedinačnih zadataka u red, i sastavljanje N rezultata u jedan izlaz nije nešto što Celery nudi kao ugrađenu mogućnost; to bi moralo biti ručno implementirano iznad Celery-ja, na način sličan onome što ovaj rad implementira koristeći Kubernetes, samo sa redom poruka i worker procesima kao izvršnim slojem umesto Kubernetes Job/Pod objekata. Drugo, a ključno za ovaj rad, broj worker-a kod Celery-ja nije direktno mapiran na broj Kubernetes pod-ova niti direktno vidljiv kroz Kubernetes API. Te bi se worker procesi, ako se izvršavaju u kontejnerima, i dalje morati orkestrisati nekim spoljnim mehanizmom, čime bi se, za potrebe ovog rada, samo prebacio isti problem orkestracije jedan nivo dublje, bez suštinske koristi.

2.1.4.Upravljani cloud servisi za paketnu obradu (AWS Batch)

AWS Batch (i analogni servisi drugih provajdera, npr. Google Cloud Batch, Azure Batch) nude potpuno upravljanu infrastrukturu za paketno izvršavanje: korisnik definiše definiciju posla (job definition, praktično Docker sliku sa komandom), red poslova i računarsko okruženje, a servis se stara o upravljanju instancama, raspoređivanju i skaliranju. Ispod haube, ovi servisi često i sami koriste kontejnersku orkestraciju (AWS Batch, na primer, može da koristi Kubernetes kao izvršni sloj), ali tu orkestraciju u potpunosti sakrivaju od korisnika.

Iz ugla ovog rada, ovakav pristup ima sledeće nedostatke. Kao najveći nedostatak, zbog potrebe ovog rada, orkestracija je u potpunosti sakrivena, pa rad ne bi mogao da demonstrira niti da meri ponašanje samog Kubernetesa kao orkestracionog sloja, što je centralni cilj rada.

2.1.5.Orkestracija tokova poslova (Apache Airflow i Argo Workflow)

Apache Airflow rešava drugačiji, iako srodan problem: orkestraciju usmerenih acikličnih grafova (Directed Acyclic Graph, DAG) međusobno zavisnih zadataka (npr. „prvo izvuci podatke, pa ih transformiši, pa ih učitaj u skladište"), a ne nužno paralelizaciju jednog homogenog zadatka nad particijama jednog skupa podataka. Airflow definiše DAG kroz Python kod (operatore povezane zavisnostima), a sam scheduler odlučuje kada je koji zadatak spreman za izvršavanje; izvršni sloj (executor) može biti lokalni proces, Celery worker, ili Kubernetes Pod (preko KubernetesExecutor-a ili KubernetesPodOperator-a).

Argo Workflows je posebno relevantan za ovaj rad jer je izgrađen kao Kubernetes-native rešenje. Svaki korak toka se izvršava kao zaseban Pod, a sam Argo kontroler je implementiran kao Kubernetes operator koji upravlja tim pod-ovima.

Nedostatak oba sistema, iz ugla problema definisanog u ovom radu, jeste da rešavaju širi i složeniji problem od onog koji je ovde postavljen: njihova osnovna jedinica je graf međuzavisnih koraka, a ne jednostavna N-way podela jednog homogenog zadatka. Njihova upotreba za ovaj, jednostavniji problem zahtevala bi uvođenje dodatnog sloja apstrakcije.

Argo Workflows je, od svih analiziranih alternativa, najbliži predloženom rešenju po filozofiji (Kubernetes-native, transparentan), ali je i dalje opšte namenski alat za orkestraciju tokova, dok predloženo rešenje ovog rada namerno ostaje minimalno i direktno oslonjeno na ugrađenu Job/Indexed completion mode semantiku, bez dodatnog operatora.

2.2. Dobre i loše strane postojecih rešenja

Tabela 2.1 sumira prethodno opisana rešenja iz ugla tri kriterijuma bitna za problem definisan u ovom radu:

- da li rešenje direktno izlaže Kubernetes primitive orkestracije (Job, Pod, rasporedjivanje) ili ih sakriva iza sopstvenog izvršnog modela
- operativnu složenost i cenu pokretanja rešenja u lokalnom, edukativnom kontekstu diplomskog rada
- da li je rešenje prilagodjeno generičkom, jednostavnom zadatku obrade po redu, ili je optimizovano za složenije scenarije (iterativni algoritmi, medjuzavisni koraci) koji za ovaj rad nisu relevantni.

Tabela 2.1

Rešenje	Izlaže K8s primitive direktno	Operativna složenost	Pogodnost za generički zadatak
Hadoop MapReduce	Ne (sopstevni klaster)	Visoka	Komplikovano
Apache Spark	Delimicno (preko K8s scheduler-a)	Srednja do visoko	Komplikovano
Celery + red poruka	Ne (sopstveni worker model)	Srednja (potreban broker: Redis/ RabbitMQ)	Dobro za asinhrone zadatke, ne za particionisanje jednog fajla
AWS Batch	Ne (potpuno sakriven)	Niska za korisnika, ali cloud-only, uz trošak i vendor lock-in	Funkcionalno
Airflow	Ne direktno	Srednja do visoka (kontroler + DB)	Nije optimalno za N-way particiju

Uporedni pregled postojećih rešenja

Iz tabele proizilazi zajednički obrazac: sistemi koji su najmoćniji za obradu podataka u opštem slučaju (Spark, Hadoop) uvode najveću operativnu složenost i najviše sakrivaju Kubernetes primitive, dok upravljani cloud servisi tu složenost prebacuju na provajdera po ceni gubitka kontrole, transparentnosti i mogućnosti lokalnog pokretanja. Sistemi za tokove poslova (Airflow, Argo) rešavaju drugačiji problem - koordinaciju medjuzavisnih koraka, te bi uveli nepotrebnu složenost za zadatak koji je, u svojoj suštini, jedna faza sa N nezavisnih paralelnih instanci.

2.3. Ciljne karakteristike rešenja

Na osnovu prethodne analize, definisane su sledeće ciljne karakteristike koje rešenje treba da ispuni:

- Automatizacija celog životnog ciklusa posla. Particionisanje ulaza, kreiranje Job-a, praćenje statusa, agregacija rezultata i čišćenje resursa, sve to bez ručne intervencije operatera nakon predaje posla.
- Merljivost i observabilnost: sistem treba da izlaže dovoljno podataka (status, vreme čekanja, vreme izvršavanja, propusnost) da omogući proveru pretpostavki o njegovom ponašanju, a ne samo funkcionalnu ispravnost.

Druga karakteristika (merljivost) nije bila eksplicitno navedena u prvobitnom predlogu teme, ali se pokazala neophodnom u ranoj fazi implementacije i postala je jedan od centralnih doprinosa rada.

2.4. Početne pretpostavke

Rešenje je razvijano pod sledećim početnim pretpostavkama, koje su eksplicitno navedene kako bi rezultati mogli da se tumače u ispravnom kontekstu:

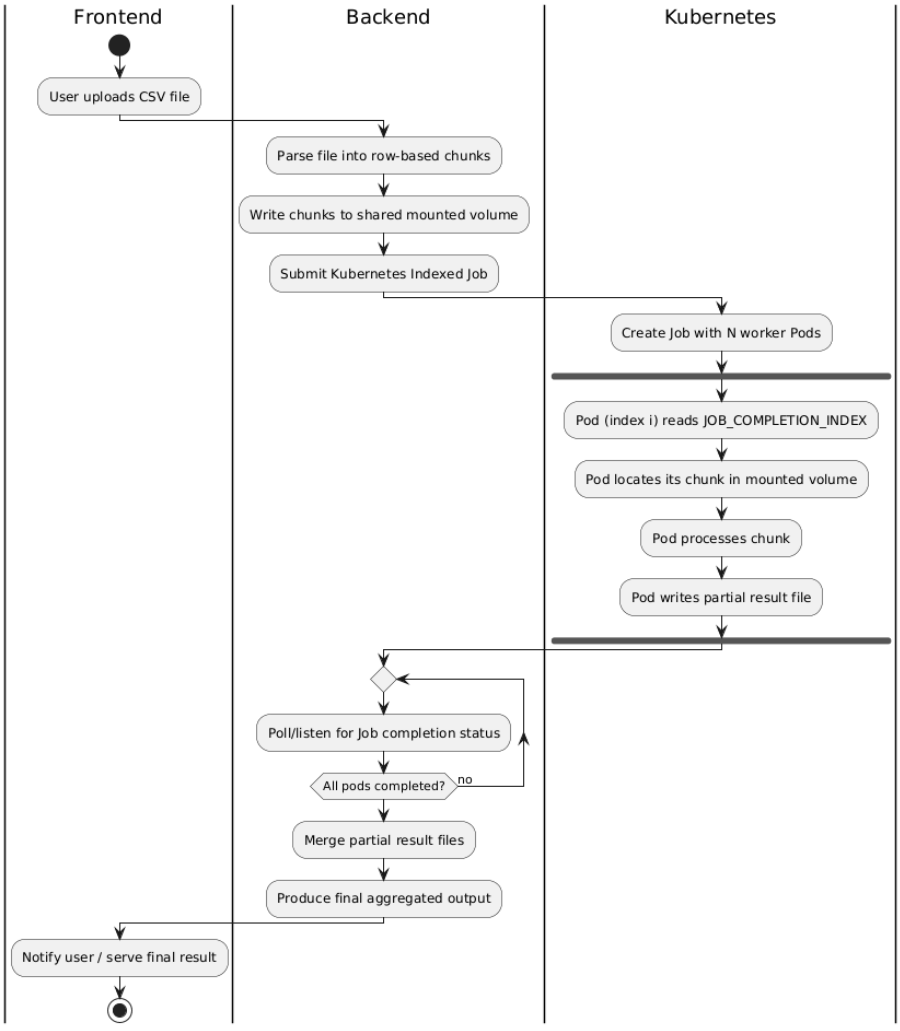
- Jednočvorni (single-node) Kubernetes klaster (minikube) dovoljan je da se demonstrira i proceni model orkestracije; zaključci o ponašanju sistema pod višečvornim (multi-node) klasterom i mrežno-priloženim skladištem su izvan obima ovog rada i navedeni su kao pravac budućeg rada.
- Korektnost obrade podataka nije predmet ispitivanja. Obrada je namerno pojednostavljena kako bi pažnja ostala na orkestracionom sloju
- Deljeno skladište zasnovano na lokalnom fajl sistemu (hostPath volumen, premošćen `minikube mount` 9p protokolom) predstavlja prihvatljivu zamenu za objektno skladište (npr. AWS S3) na razmerama koje se testiraju u ovom radu (do 10 miliona redova), uz svest da ovakvo rešenje suštinski ne funkcioniše na višečvornom klasteru.
- Ulazne CSV datoteke se u najvećoj meri pretpostavljaju strukturno ispravnim, uz validaciju na nivou otpremanja, s obzirom da korektnost sadržaja podataka nije fokus rada.
- Broj worker-a po poslu je ograničen maksimumom od 32, izabranim kao bezbednosna granica za jednočvorni minikube klaster sa ograničenim brojem CPU jezgara, a ne kao ograničenje predloženog modela.

Ove pretpostavke nisu naknadno pronadjena opravdanja za ograničenja implementacije, već su formulisane pre početka realizacije, kao deo definisanja obima rada i predloga teme. Formulacija pretpostavki omogućava da se rezultati čitaju u ispravnom kontekstu (npr. da zaključak o I/O-ograničenom ponašanju sistema važi za konfiguraciju sa hostPath skladištem, a ne ogranicenje Kubernetesa) i jasno razgraničava da je u ovom radu predmet ispitivanja model orkestracije i njegovo ponašanje pod paralelizacijom.

2.5. Predlog rešenja problema na globalnom nivou

Na osnovu ciljnih karakteristika i početnih pretpostavki, predloženo je rešenje čija je arhitektura, na globalnom nivou, sledeći lanac komponenti:

Frontend (React) → Backend API (FastAPI) → Particioner (Partitioner) → Kubernetes (Job/ Pod orkestracija) → Worker-i (kontejnerizovani izvršioci) → Deljeno skladište → Agregator (Aggregator) → Frontend (prikaz/preuzimanje rezultata).



Slika 1: Dijagram aktivnosti

Tok podataka i kontrole kroz sistem, od otpremanja CSV datoteke do preuzimanja agregiranog rezultata

Korisnik otprema CSV datoteku i bira broj worker-a preko frontend-a. Backend validira ulaz, snima ga u privremeno skladište i po zahtevu za pokretanje particioniše ulaznu datoteku na onoliko delova (chunk-ova) koliko je odabrano worker-a, a zatim predaje Kubernetesu jedan Job objekat sa Indexed completion mode-om i N pod-ova. Svaki pod, na osnovu sopstvenog rednog broja koji mu Kubernetes dodeljuje preko API-ja, čita tačno onaj deo ulaznih podataka koji mu pripada, izvršava (simuliranu, u okviru obima rada) obradu i upisuje svoj parcijalni rezultat i podatke o vremenu izvršavanja u deljeno skladište. Backend prati status Job-a kroz kombinaciju Kubernetes watch mehanizma (za trenutačno reagovanje) i periodičnog poll-on-read mehanizma (kao rezervni put), a po završetku posla agregira sve parcijalne rezultate u jedan izlazni fajl i čisti privremene Kubernetes i fajl-sistemske resurse.

Ovako definisano rešenje ne uvodi sopstveni izvršni sloj iznad Kubernetesa, već Kubernetes Job jeste izvršni sloj. Za razliku od AWS Batch-a i sličnih upravljanih servisa, ne zahteva plaćeni cloud nalog i u potpunosti je transparentan (svaki korak se može posmatrati preko kubectl-a); za razliku od Airflow-a/Argo-a, ne uvodi koncept medjuzavisnih koraka tamo gde ne postoji potreba za njim (zadatak je jedna faza sa N nezavisnih instanci).

3. Pregled detalja rešenja

Prilikom realizacije sistema korišćen je sledeći skup tehnika i obrazaca:

- Kontejnerizacija (Docker) - backend i frontend razvojno okruženje i worker komponenta koja se izvršava unutar Kubernetes pod-ova pakovani su kao Docker slike, čime se obezbedjuje da se izvršno okruženje worker-a ne razlikuje od onoga u kom je testirano
- Orkestracija preko Kubernetes Jobs API-ja, sa Indexed completion mode-om za dodelu rednog broja svakom pod-u
- Dizajn REST API-ja (FastAPI, sa Pydantic šemama za validaciju zahteva i odgovora)
- Perzistencija stanja posla preko ORM sloja (SQLAlchemy) nad SQLite bazom
- Asinhrono pozadinsko izvršavanje (asyncio pozadinski task pokrenut u okviru FastAPI lifespan mehanizma) za watch stream, uz threading.Lock za sinhronizaciju pristupa stanju posla izmedju watch i poll putanja
- Protočna (streaming) obrada ulazno-izlaznih fajlova u modulima za particionisanje i agregaciju, kako veličina ulaznog skupa podataka (do 10 miliona redova) ne bi zahtevala učitavanje cele datoteke u memoriju
- Instrumentacija vremena izvršavanja na nivou pojedinačnog pod-a

3.1. Kubernetes Indexed Job i API mehanizam

Pošto je Indexed completion mode centralni mehanizam čitavog rešenja, njegovo tačno ponašanje je ovde detaljnije objašnjeno. Kada se Kubernetes Job kreira sa `completionMode: Indexed` i `completions: N`, `parallelism: N`, Kubernetes kontroler za Job resurse kreira N pod-ova, i svakom od njih, preko takozvanog downward API mehanizma (mehanizma kojim Kubernetes pod-u prosledjuje podatke o sopstvenom okruženju, a da pod ne mora sam da poziva Kubernetes API), dodeljuje jedinstven redni broj od 0 do N-1.

Ovaj redni broj je dostupan pod-u na dva ekvivalentna načina:

- kao vrednost oznake (labela) `batch.kubernetes.io/job-completion-index` na samom pod-u
- kao unapred postavljena promenljiva okruženja `JOB_COMPLETION_INDEX`, koju worker u ovom sistemu direktno čita (u novijim verzijama alata)

Ova osobina je presudna za arhitekturu opisanu u ovom radu jer omogućava da worker, bez ikakve dodatne komunikacije sa backend-om ili Kubernetes API-jem, samostalno „zna" koji deo posla mu pripada.

Dovoljno je da backend, prilikom particionisanja, imenuje chunk fajlove po istoj šemi koju worker koristi da izračuna svoje ime fajla (`chunk_{JOB_COMPLETION_INDEX}.csv`). Bez Indexed completion mode-a, backend bi morao ili da kreira N zasebnih Pod specifikacija (svaku sa hardkodovanim brojem chunk-a kroz promenljivu okruženja ili argument komandne linije), ili da worker-ima dodeli chunk-ove preko nekog drugog kanala komunikacije, od kojih bi svaki uveo dodatnu komponentu stanja i dodatnu tačku otkaza. Indexed completion mode svodi celu ovu koordinaciju na jedno svojstvo Kubernetes Job objekta, čime particionisanje i orkestracija ostaju potpuno odvojeni. Particioner ne mora ništa da zna o Kubernetesu, a Kubernetes ne mora ništa da zna o sadržaju particija, sve dok se obe strane drže iste konvencije o imenovanju fajlova.

3.2. Najbitniji elementi realizacije resenja

Od svih realizovanih mehanizama, sledeća četiri izdvajaju se kao centralni za razumevanje sistema, jer direktno odredjuju njegovu ispravnost, pouzdanost i merljivo ponašanje:

1. Jedan Indexed Kubernetes Job kao jedinica izvršavanja posla, umesto N zasebnih Job/ Pod specifikacija. Svaki posao korisnika mapira se na tačno jedan Kubernetes Job objekat, čime se broj Kubernetes API poziva i složenost upravljanja svode na minimum
2. Dvostruki način ažuriranja statusa. Kombinacija Kubernetes watch stream-a (reakcija na dogadjaje u realnom vremenu) i poll-on-read mehanizma (rezervni put, otporan na propuštene dogadjaje usled restarta backend-a)
3. Proporcionalni rok izvršavanja (activeDeadlineSeconds), izračunat na osnovu broja redova po worker-u, koji rešava problem poslova čiji pod-ovi nikada ne dostignu terminalnu fazu
4. Instrumentacija vremena po pod-u i poseban „noop-transform" režim rada worker-a, koji zajedno omogućavaju da se razdvoji doprinos ulazno-izlaznih operacija od doprinosa izračunavanja u ukupnom vremenu izvršavanja posla

3.3. Uslovi pod kojima se rešenje primenjuje

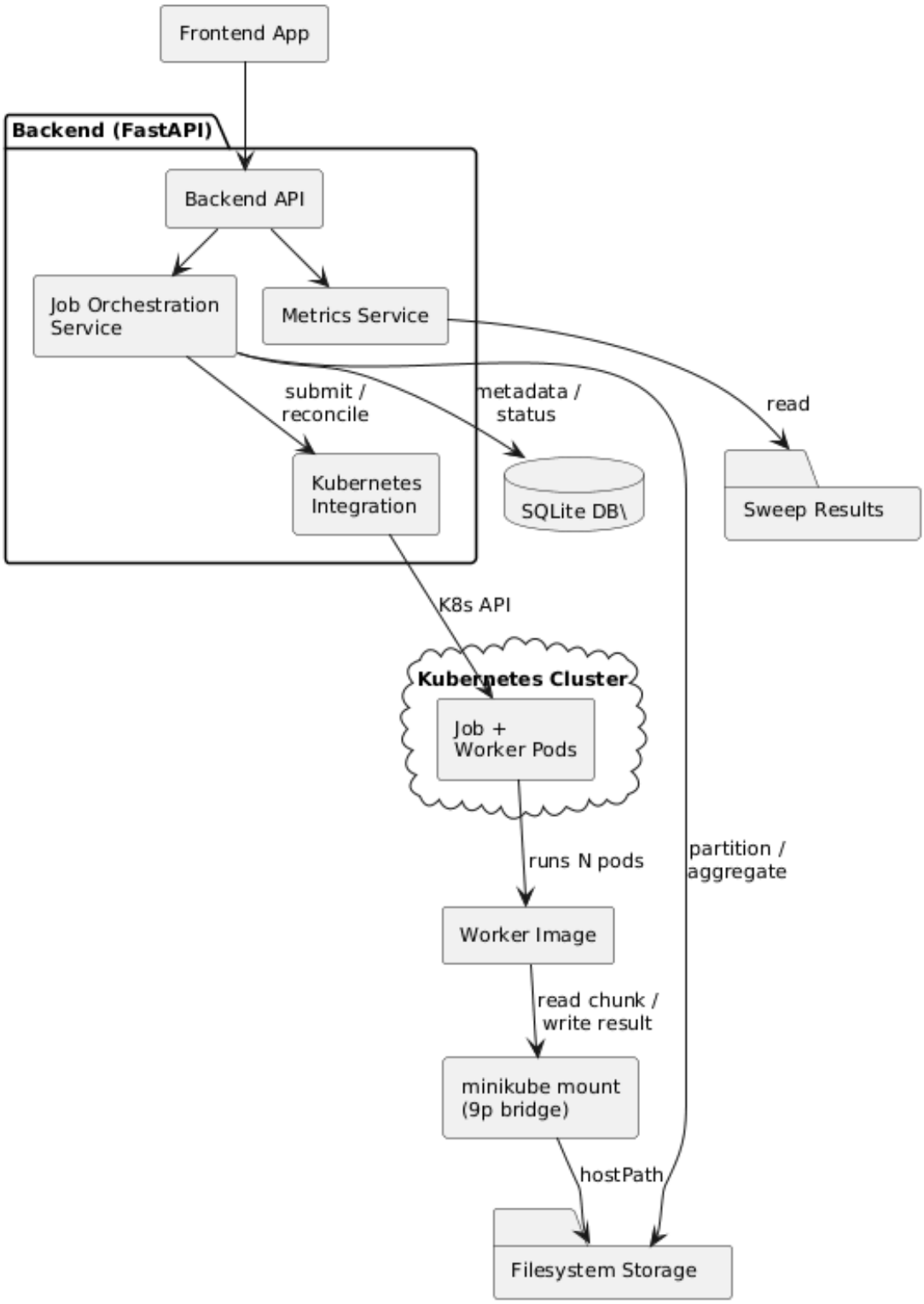
Sistem je razvijan i testiran u sledećem okruženju, čije je poznavanje neophodno za tumačenje implementacije i rezultata:

- Lokalni Kubernetes klaster pokrenut preko minikube-a (host mašina pokreće minikube VM/kontejner preko Docker-a)
- Jednočvorni klaster, čiji se raspoloživi kapacitet CPU/RAM resursa proverava komandom `kubectl describe node minikube` pre nego što se posao sa velikim brojem worker-a pokrene, kako bi se izbeglo da pod-ovi ostanu u stanju Pending zbog nedovoljnih resursa. Kubernetes ne odbija Job koji se ne može rasporediti u trenutku predaje, već pod-ovi jednostavno ostaju Pending, pa je backend-u dodata sopstvena provera kapaciteta pre predaje Job-a
- Backend zahteva Python 3.11 i virtuelno okruženje (venv) sa instaliranim zavisnostima (FastAPI, SQLAlchemy, kubernetes klijent za Python, uvicorn kao server); frontend zahteva Node.js okruženje i npm za pokretanje Vite razvojnog servera
- Deljeno skladište izmedju host fajl sistema i Kubernetes klastera realizovano je preko hostPath volumena, premošćenog procesom `minikube mount backend/storage:/data`, koji mora biti pokrenut i aktivan tokom čitavog izvršavanja bilo kog posla. Ovo je zaseban, dugotrajni proces koji korisnik mora ručno održavati
- Worker slika koja gradi se lokalno, Docker build komandom, a zatim se učitava u minikube klaster

4. Implementacija rešenja

Sistem izvršava sledeći tok, ilustrovan dijagramom aktivnosti na Slici 1 i dijagramom komponenti na Slici 2:

- Korisnik, preko frontend forme za otpremanje bira CSV datoteku i broj worker-a, i šalje zahtev `POST /api/jobs`. Backend validira strukturu datoteke (prisustvo zaglavlja, dosledan broj kolona po redu, neprazan sadržaj), snima datoteku na `storage/jobs/{id}/input.csv`, upisuje red u bazu sa statusom Pending i vraća identifikator posla.
- Korisnik pokreće posao klikom na dugme "Run" (u pozadini se pokreće `POST /api/jobs/{id}/run`). Backend particioniše ulaznu datoteku (modul partitioner) u onoliko chunk fajlova koliko je odabrano worker-a, koristeći već poznat broj redova (izračunat u koraku 1) kako bi ravnomerno rasporedio redove (uz raspodelu ostatka pri deljenju), zatim gradi Kubernetes Job manifest (modul job_builder) sa `completions = parallelism = worker_count`, `completionMode: Indexed`, `backoffLimit: 0` i izračunatim `activeDeadlineSeconds` i predaje ga Kubernetes API-ju (modul orchestrator). Status posla prelazi u Running.
- Kubernetes kreira N pod-ova. Svaki pod, preko downward API-ja, čita sopstveni redni broj (JOB_COMPLETION_INDEX), na osnovu njega izračunava ime sopstvenog chunk fajla (`chunk_{index:03d}.csv`), čita ga preko deljenog volumena, izvršava obradu po redu, i upisuje rezultat (`result_{index:03d}.csv`) i podatke o vremenu izvršavanja (`timing_{index:03d}.json`) natrag u deljeno skladište, uz eksplicitni fsync (neophodan jer je skladište premošćeno preko protokola 9p koji ne garantuje trajnost bez eksplicitnog sinhronizovanja).
- Backend prati status posla dvostrukim mehanizmom (watcher + poll-on-read). Kada Kubernetes Job dostigne stanje Complete, backend agregira sve `result_*.csv` fajlove u jedan `output.csv` (modul aggregator) i status posla prelazi u Completed; ako Job dostigne stanje Failed, backend beleži poruku greške i status prelazi u Failed. U oba slučaja, neposredno pre nego što se pod-ovi obrišu, snima se snapshot statusa svakog worker pod-a (worker_snapshot), kako bi detaljan prikaz posla u frontend-u i dalje imao šta da prikaže i nakon što Kubernetes obriše same pod-ove.
- Odmah po dostizanju terminalnog stanja (Completed ili Failed), pokreće se automatsko čišćenje: Kubernetes Job se briše (sa `propagationPolicy: Background`, što kaskadno briše i njegove pod-ove), a direktorijumi `chunks/` i `results/` se uklanjaju sa fajl sistema; `input.csv` i `output.csv` se zadržavaju.
- Korisnik preuzima rezultat preko `GET /api/jobs/{id}/result`



Slika 2: Dijagram komponenti sistema

Frontend tokom čitavog procesa periodično (na svake 3 sekunde) osvežava prikaz liste poslova preko `GET /api/jobs`, uz optimizaciju da se osvežavanje pauzira kada je tab u pozadini ili kada su svi poslovi u terminalnom stanju, i odmah nastavlja kad tab ponovo postane vidljiv ili kad korisnik sam pokrene novi posao.

Sistem je organizovan u tri nezavisne celine: backend, worker i frontend čija je fajl struktura prikazana u Tabeli 3.2 (skraćeni prikaz; potpuni raspored dat je u Prilogu 7.1).

Tabela 3.2

Modul	Odgovornost
main.py	Inicijalizacija FastAPI aplikacije, CORS podešavanja, montiranje rutera, kreiranje tabela i pokretanje/zaustavljanje pozadinskog watch task-a preko lifespan mehanizma
core/config.py	Konfiguracija sistema (Settings), sa mogućnošću promene svih parametara preko promenljivih okruženja
core/db.py	Podešavanje SQLAlchemy sesije
models/job.py	ORM model posla (Job) i enumeracija statusa (JobStatus)
schemas/job.py	Pydantic šema odgovora (JobRead), uključujući izvedene metrike (vreme čekanja, vreme izvršavanja, propusnost)
schemas/metrics.py	Pydantic šeme za benchmark metrike (SweepRun, SweepPoint)
api/jobs.py	Svi /api/jobs* endpoint-i
api/metrics.py	Endpoint-i /api/metrics/sweep(/runs)
services/job_service.py	Centralna orkestraciona logika: kreiranje, pokretanje, particionisanje, agregacija, čišćenje, rekoncilijacija (pojedinačna i grupna), primena statusa
services/partitioner.py	Particionisanje ulazne CSV datoteke
services/aggregator.py	Spajanje parcijalnih rezultata u jedan izlaz
services/ metrics_service.py	Čitanje sweep_results/*.csv fajlova i izvođenje metrika ubrzanja/ efikasnosti/propusnosti
k8s/client.py	Učitavanje kubeconfig-a i keširane fabrike API klijenata
k8s/job_builder.py	Konstrukcija V1Job manifesta (Indexed completion mode, hostPath volumen, izračunavanje roka po poslu)
k8s/orchestrator.py	Omotač nad kubernetes klijentom: predaja, čitanje, listanje (grupno), brisanje Job-a; listanje worker pod-ova
k8s/watcher.py	Pozadinski asyncio task koji prati Kubernetes Job događaje preko watch mehanizma i prosleđuje ih rekoncilijacionoj logici

Struktura backend modula (backend/app/)

Od svih modula prikazanih u Tabeli 3.2, četiri zaslužuju detaljniji opis, jer zajedno realizuju centralne elemente sistema:

- `services/job_service.py` je najveći i konceptualno najvažniji modul backend-a. Predstavlja jedinu tačku kroz koju prolazi bilo koja promena stanja posla, bilo da je pokrenuta korisničkom akcijom (kreiranje, pokretanje posla) ili spoljnim dogadjajem (watch, poll). Pored funkcija `create_job` i `run_job`, modul izlaže tri funkcije za azuriranje stanja sa različitom granulacijom: `reconcile_status` za osvežavanje jednog posla (poziva se kad korisnik otvori detalje jednog posla), `reconcile_jobs_batch` za grupno osvežavanje (poziva se pri listanju svih poslova, i interno izvršava tačno jedan Kubernetes API poziv nezavisno od broja poslova koji čekaju osvežavanje), i `reconcile_from_k8s_job`, koju poziva watch mehanizam direktno sa već pristiglim Kubernetes Job objektom. Sve tri funkcije, na kraju svog toka, pozivaju zajedničku `apply_k8s_job_status` funkciju, čime je zagarantovano da se logika prelaza stanja (uključujući agregaciju i čišćenje) ne duplira na tri mesta i da se ne sistem ne menja u zavisnosti od toga koji je put doveo do azuriranja stanja.
- `k8s/watcher.py` implementira pozadinski zadatak koji se pokreće jednom, pri podizanju backend aplikacije i radi tokom čitavog veka trajanja procesa. Unutar njega, `run_job_watcher` funkcija u beskonačnoj petlji otvara `kubernetes.watch.Watch()` stream filtriran oznakom koja identifikuje Job objekte ovog sistema, i za svaki primljeni dogadjaj poziva `_handle_event`, koja iz oznake Job objekta izvlači identifikator posla (`_job_id_from_k8s_job`) i prosledjuje ga metodi `job_service.reconcile_from_k8s_job`. Watch konekcija se, u praksi, s vremena na vreme prekida (istek vremena na strani Kubernetes API servera, mrežni problem) te `_stream_events` funkcija to tretira kao očekivan, a ne izuzetan dogadjaj
- `k8s/job_builder.py` je zadužen za konstrukciju samog Kubernetes Job manifesta. Metoda `build_job_manifest` sastavlja specifikaciju sa `completions`, `parallelism`, `completionMode: Indexed`, hostPath volumenom montiranim na `data_mount_path`, promenljivama okruženja koje se prosledjuju svakom pod-u (uključujući COMPUTE_FIXED_S / COMPUTE_PER_ROW_S argumente, koje omogućavaju benchmark eksperiment bez izmene worker slike), i izračunatim `activeDeadlineSeconds` preko `_active_deadline_seconds` funkcije. Ovaj modul takodje čuva zajedničke konstante (oznaku posla `JOB_ID_LABEL` i odgovarajući selektor `JOB_LABEL_SELECTOR`) koje koriste i `orchestrator.py` (za listanje/brisanje) i `watcher.py` (za filtriranje watch stream-a), čime je garantovano da svi delovi sistema „gledaju" iste Job objekte.
- `k8s/orchestrator.py` je jednostavan omotač nad kubernetes Python klijentom. Svaka njegova funkcija (`submit_job`, `get_k8s_job`, `list_k8s_jobs`, `list_worker_pods`, `delete_k8s_job`) odgovara jednom logičkom Kubernetes API pozivu, uz jednu bitnu razliku u tretmanu grešaka: modul pravi razliku izmedju situacije u kojoj Kubernetes API vrati 404 i situacije u kojoj je klaster potpuno nedostupan (podiže se posebna `KubernetesUnavailableError`), što `job_service.py` koristi da razlikuje „posao čiji Job objekat je uklonjen" od „trenutno ne mogu da proverim stanje klastera".Potrebna distinkcija kako ne se ne bi pogrešno proglasio posao neuspelim samo zato što je Kubernetes API server privremeno nedostupan.

Worker komponenta (worker/main.py) je samostalan Python skript, koji se izvršava unutar svakog pod-a: čita svoje parametre iz promenljivih okruženja (JOB_ID, JOB_COMPLETION_INDEX,

DATA_ROOT, COMPUTE_FIXED_S, COMPUTE_PER_ROW_S), čita svoj chunk, izvršava transformaciju po redu i upisuje rezultat i podatke o vremenu.

Frontend (frontend/src/) je React aplikacija (Vite build alat, Tailwind CSS za stilizovanje, Recharts biblioteka za grafikone), organizovana oko komponenata App.jsx (vrhovno stanje, petlja osvežavanja, navigacija izmedju Jobs i Metrics tabova), api.js (omotači nad fetch pozivima ka backend API-ju), UploadForm.jsx, JobList.jsx, JobDetailModal.jsx (prikaz statusa pojedinačnih worker pod-ova, sa sopstvenim osvežavanjem dok je modal otvoren), StatusBadge.jsx, PipelineOverview.jsx (statički dijagram od pet koraka), JobDurationChart.jsx (grafikon vremena izvršavanja po broju worker-a, iz stvarnih, organski prikupljenih podataka o završenim poslovima) i MetricsPanel.jsx (grafikoni izvedeni iz benchmark sweep podataka).

U nastavku su prikazane ilustrativne reprezentacije (pseudokod) logike karakterističnih delova sistema, radi preciznijeg objašnjenja njihovog ponašanja.

- Proporcionalni rok izvršavanja posla (k8s/job_builder.py):

```
def _active_deadline_seconds(rows_per_worker):
    return job_deadline_base_seconds + job_deadline_seconds_per_row * ceil(rows_per_worker)
    # podrazumevano: base = 60 [s], per_row = 0.5 [s]
```

Ova formula obezbeđuje da rok raste sa količinom posla koji je dodeljen pojedinačnom worker-u (rows_per_worker = row_count / worker_count), umesto da bude fiksna konstanta ista za sve poslove.

- Primena statusa iz Kubernetes Job objekta, sa serijalizacijom preko brave po poslu (services/job_service.py), koja se poziva i iz watch i iz poll putanje:

```
def apply_k8s_job_status(job_id, k8s_job):
    with lock_for(job_id):
        if job.status != Running:
            return  # već je u terminalnom stanju
        if k8s_job.status.failed_condition:
            job.error_message = k8s_job.status.condition_message
            job.status = Failed
        elif k8s_job.status.complete_condition:
            snapshot_worker_statuses(job_id)
            aggregate_csv(job_id)
            job.status = Completed
        if job.status in (Failed, Completed):
            job.finished_at = now()
            cleanup_job(job_id)
```

Provera `if job.status != Running: return` na početku funkcije, pod bravom, jeste mehanizam koji sprečava da watch događaj i poll osvežavanje istog posla, ako se dogode gotovo istovremeno, dvaput izvrše agregaciju ili čišćenje.

- Model simulirane cene izračunavanja u worker-u (worker/main.py), korišćen u benchmark eksperimentu (poglavlje 4):

```
def simulated_compute(rows_in_chunk, noop_transform):
    if noop_transform:
        return  # preskače se za I/O-only merenje
    time.sleep(compute_fixed_s + compute_per_row_s * rows_in_chunk)
    # podrazumevano: compute_fixed_s = 0.3 [s], compute_per_row_s = 0.000001 [s]
```

4.1. Opis donetih projektnih odluka

Nekoliko odluka donetih tokom realizacije koje je vredno naknadno obrazložiti. One su ovde eksplicitno navedene, uz obrazloženje.

- Jedan Indexed Job umesto N zasebnih Job/Pod specifikacija. Alternativa predstavlja kreiranje N nezavisnih Kubernetes Job objekata, po jednog za svaki worker ili N Pod specifikacija direktno. Bila bi jednostavnija za razumevanje na prvi pogled, ali bi zahtevala ili N poziva Kubernetes API-ja pri predaji posla, ili ConfigMap mehanizam za distribuciju liste zadataka po pod-u, i N nezavisnih objekata za praćenje statusa i čišćenje. Indexed completion mode rešava ovo elegantno: jedan Job objekat, sa `completions = parallelism = worker_count`, dodeljuje svakom pod-u redni broj preko downward API-ja, koji pod koristi da izračuna svoje ime chunk/rezultat fajla
- hostPath + `minikube mount` umesto ConfigMap-a i čitanja rezultata iz logova pod-ova. Razmotrena je i alternativa u kojoj bi se chunk podaci prosleđivali kroz ConfigMap objekte, a rezultati čitali iz standardnog izlaza (stdout) pod-ova, čime bi se u potpunosti izbegla zavisnost od mount mehanizma
- Dvostruko (watch + poll) azuriranje statusa umesto isključivo watch mehanizma. Čist watch pristup nije dovoljan sam po sebi jer watch konekcija vidi samo događaje koji se dese od trenutka kada je otvorena (u slucaju da se backend restartuje, propustiće sve događaje koji su se desili dok je bio ugašen). Iz tog razloga, svaki `GET /api/jobs` ili `GET /api/jobs/{id}` i dalje azurira poslove u stanju Running kao rezervni put, s tim da se za listu poslova ovo radi kao jedan poziv `list_namespaced_job` za sve poslove u stanju Running odjednom (`reconcile_jobs_batch`), a ne jedan poziv po poslu.
- Proporcionalan, a ne fiksan rok izvršavanja (activeDeadlineSeconds). Testiranje je pokazalo da posao čiji pod-ovi nikada ne dostignu terminalnu fazu (npr. `ImagePullBackOff`) nema drugi put do stanja Failed. Job prelazi u Failed samo kada broj stvarnih neuspeha pod-ova premaši `backoffLimit`, a ne zbog pod-ova koji samo stoje zaglavljeni u stanju Pending. Prvobitno rešenje (fiksnih 300 sekundi za sve poslove) je zamenjeno formulom `60 + 0.5 × broj_redova_po_worker-u` [sekund], jer bi fiksan rok prisilno prekinuo veliki, ali zdrav posao kao i mali ali zaglavljen posao. I dalje je reč o fiksnoj vrednosti koja je usvojena kao takva bez ikakvih dokaza
- SQLite umesto PostgreSQL-a za metapodatke o poslovima. Prvobitni predlog teme pominjao je PostgreSQL kao kandidata za bazu podataka. Ovo odstupanje je svesno prihvaćeno jer sistem, u skladu sa definisanim obimom rada, nema autentifikaciju niti potrebu za izolacijom podataka po korisniku, i pokreće se kao jedna instanca backend procesa

- Gornja granica od 32 worker-a po poslu (max_workers_per_job) je promenljiva bezbednosna granica prilagođena minikube klasteru sa ograničenim brojem CPU jezgara, a ne ograničenje samog Kubernetes-a

4.2. Mogućnosti za modifikaciju rešenja

Arhitektura je namerno projektovana sa jasnim granicama između slojeva (particionisanje, orkestracija, agregacija, metrike), što omogućava proširenje modula, bez potrebe za promenom osnovne arhitekture:

- Zamena hostPath/`minikube mount` skladišta čime bi se uklonila zavisnost od 9p mosta i omogućilo rad na višečvornom klasteru.
- Uvođenje interfejsa za obradu (validate/partition/execute) koji bi generalizovao particionisanje sa CSV-a specifičnog pristupa na proizvoljan tip ulaza (TXT, JSON, slike, logovi), pri čemu bi svaki tip ulaza imao sopstveni particioner, aorkestracioni sloj (Job builder, watcher, rekoncilijacija) ostao nepromenjen.
- Ograničen broj ponovnih pokušaja (`backoffLimit > 0`) za privremene otkaze pojedinačnog pod-a (npr. neuspešno preuzimanje slike), uz zadržavanjedeterminističke dodele chunk-a preko completion index-a, tako da ponovljeni pod obrađuje isti, a ne drugi deo podataka.
- Izvođenje konstante u formuli za rok izvršavanja (job_deadline_seconds_per_row) iz stvarno izmerene propusnosti sistema, umesto iz fiksne, ručno podešene vrednosti.
- Premeštanje sa SQLite na PostgreSQL, ako bi se sistem proširio na više istovremenih instanci backend-a.
- Dodavanje autentifikacije, upravljanja resursima po poslu (CPU/RAM) i podrške za dodatne formate ulaza (ZIP)

4.3. Korisničko uputstvo

Tabela 3.3 prikazuje kompletan API sistema, koji ujedno predstavlja i skup funkcija koje sistem nudi korisniku (preko frontend-a) ili programski (preko direktnih HTTP poziva).

Tabela 3.3

Model	Putanja	Funkcija	Odgovor greške
POST	/api/jobs	Otpremanje CSV datoteke i broja worker-a, kreiranje posla	422: neispravan produžetak, prazan/ samo-zaglavlje/ nepravilan CSV, worker_count van granica ili veći od broja redova
GET	/api/jobs	Lista svih poslova (prethodno rekoncilira poslove u stanju Running)	-
GET	/api/jobs/{id}	Podaci o jednom poslu (ažurira status ako je Running)	404

Model	Putanja	Funkcija	Odgovor greške
POST	/api/jobs/{id}/run	Particionisanje i predaja posla Kubernetesu	404; 409 ako posao nije u stanju Pending (uključujući slučaj dvostruke predaje)
GET	/api/jobs/{id}/result	Preuzimanje agregirane izlazne datoteke	404; 409 ako posao nije u stanju Completed
GET	/api/jobs/{id}/ workers	Status pojedinačnih worker pod-ova posla	-
GET	/api/health	Provera dostupnosti servisa	-
GET	/api/metrics/sweep/ runs	Lista dostupnih (mode, broj_redova) parova benchmark rezultata	-
GET	/api/metrics/sweep	Izvedene metrike (ubrzanje, efikasnost, propusnost) za jedan benchmark run	404: run ne postoji; 422: run postoji ali nema tačku sa worker_count=1 potrebnu za računanje ubrzanja

API sistema

4.3.1.Primer tipičnog scenarija korišćenja

Koraci koje korisnik prati da bi obradio CSV datoteku:

1. Otvoriti frontend aplikaciju (podrazumevano na http://localhost:5173 ili sledećem slobodnom portu koji Vite dodeli)
2. U formi za otpremanje odabrati CSV datoteku i uneti željeni broj worker-a (celobrojna vrednost između 1 i konfigurisanog maksimuma, podrazumevano 32, i ne veća od broja redova u datoteci)
3. Potvrditi otpremanje - posao se pojavljuje u listi poslova sa statusom Pending
4. Pokrenuti posao dugmetom. Status posla prelazi u Running, a detaljan prikaz posla (klikom na posao) pokazuje status pojedinačnih worker pod-ova, osvežavan na svake 3 sekunde dok je otvoren
5. Sačekati da status pređe u Completed (ili Failed, uz prikaz poruke greške ako je do otkaza došlo)
6. Preuzeti rezultat dugmetom za preuzimanje, koje šalje zahtev ka `GET /api/jobs/{id}/result` i preuzima `output.csv` datoteku.
7. (Opciono) Preći na Metrics tab za pregled grafikona izvedenih iz benchmark sweep podataka, ako su takvi podaci prethodno generisani

Pored ovog, osnovnog scenarija, korisnik se može naći i u sledeće dve situacije, čiji je tok kroz interfejs vredno posebno opisati:

- Scenario „posao neuspešno završen”: ako posao pređe u stanje Failed (npr. usled neispravne slike,nepokrenutog servisa mount), status u listi poslova menja boju i oznaku, a otvaranjem detalja posla korisnik vidi tekst poruke greške preuzet direktno iz Kubernetes Job objekta (`error_message` polje) i snimak poslednjeg poznatog statusa pojedinačnih worker pod-ova (`worker_snapshot`), koji ostaje dostupan i nakon što su stvarni pod-ovi već obrisani automatskim čišćenjem. Dugme za preuzimanje rezultata u ovom stanju nije aktivno (odgovara 409 grešci API-ja, Tabela 3.3), pošto agregacija nikad nije izvršena. Korisnik u ovom slučaju otprema istu ili ispravljenu datoteku kao novi posao.
- Scenario „pregled istorijskih metrika performansi”: nakon što je benchmark sweep skripta prethodno izvršena i generisala rezultate u `backend/scripts/sweep_results/`, korisnik u Metrics tabu vidi listu dostupnih kombinacija (režim, broj_redova) preko `GET /api/metrics/sweep/runs`, a izborom jedne ili više njih učitava odgovarajuće grafikone (Slika 4). Ovaj tab je nezavisan od trenutnog stanja bilo kog pojedinačnog posla. Prikazuje istorijske, već izračunate podatke, i ne zahteva da klaster bude aktivan da bi se pregledao, sve dok backend proces (koji čita CSV fajlove sa diska) radi.

4.3.2.Opis izvornog koda i tehnički detalji za izgradnju izvršnog koda

Repozitorijum je organizovan u tri nezavisna poddirektorijuma: `backend/`, `worker/` i `frontend/`. Backend zahteva Python 3.11 i zavisnosti navedene u `requirements.txt` (FastAPI, uvicorn, SQLAlchemy, kubernetes klijent za Python, pydantic-settings i pytest za testiranje), instalirane u virtuelno okruženje (`python -m venv .venv`, `pip install -r requirements.txt`). Worker slika se gradi standardnom Docker build komandom nad `worker/Dockerfile`, koji koristi `python:3.12-slim` baznu sliku i ne uvodi dodatne zavisnosti van standardne biblioteke, čime je slika minimalna po veličini i površini za potencijalne bezbednosne propuste. Frontend zahteva Node.js okruženje i instalaciju zavisnosti preko `npm install`, nakon čega se razvojni server pokreće preko `npm run dev` (Vite), a produkcioni build preko `npm run build`.

4.3.3.Format ulaznih i izlaznih datoteka

Ulazna datoteka mora biti u CSV formatu, sa zaglavljem u prvom redu i doslednim brojem kolona u svim narednim redovima. Interni fajlovi koje sistem generiše po poslu, u direktorijumu `storage/jobs/{id}/`, prikazani su u Tabeli 3.4.

Tabela 3.4

Fajl	Sadržaj
input.csv	Originalna otpremljena datoteka (zadržava se i nakon završetka posla)

Fajl	Sadržaj
chunks/chunk_{i:03d}.csv	Deo ulaznih podataka dodeljen worker-u sa rednim brojem i (brisan nakon čišćenja)
results/result_{i:03d}.csv	Parcijalni rezultat worker-a i (brisan nakon čišćenja)
timings/timing_{i:03d}.json	Vremenski podaci worker-a i: {worker_index, rows, noop_transform, read_s, compute_s, write_s, total_s} (nikada se ne briše automatskim čišćenjem, pa preživljava završetak posla)
output.csv	Agregirani konačan rezultat (zadržava se nakon završetka posla)

Interni format fajlova po poslu

Rezultati benchmark sweep-a upisuju se u zaseban CSV po kombinaciji (režim, broj_redova) u `backend/scripts/sweep_results/`, sa kolonama: worker_count, row_count, wall_s, avg_read_s, avg_compute_s, avg_write_s, max_read_s, max_compute_s, max_write_s.

4.3.4. Detaljan opis instalacije i pokretanja

Redosled koraka za pokretanje sistema iz stanja u kome su svi procesi zaustavljeni (npr. nakon ponovnog pokretanja razvojne mašine):

1. Pokrenuti minikube klaster: `minikube start` (vraća prethodno kreiran klaster u aktivno stanje, uključujući prethodno učitanu worker sliku, bez ponovnog izvršavanja početnog podešavanja).
2. Pokrenuti most za deljeno skladište, u zasebnom terminalu/pozadinskom procesu, i ostaviti ga aktivnim tokom celog rada sa sistemom: `minikube mount backend/storage:/data`. Ovaj korak je preduslov bez kog nijedan posao ne može biti uspešno izvršen, ako proces prestane da radi usred posla, pod-ovi koji pokušaju da pristupe svom chunk-u ili upišu rezultat neuspevaju sa greškom tipa FileNotFoundError. Posebno je važno napomenuti da most može postati neispravan a da pri tom sâm proces ostane pokrenut (tzv. „stale" stanje), 9p sesija ka virtuelnoj mašini prestane da radi dok host proces i dalje izgleda aktivan, pa je za pouzdanu proveru neophodno stvarno pokušati upis/čitanje kroz most (npr. preko privremenog pod-a), a ne samo proveriti da li je proces živ.
3. Pokrenuti backend, iz `backend/` direktorijuma sa aktiviranim virtuelnim okruženjem: `uvicorn app.main:app --reload`.
4. Pokrenuti frontend, iz `frontend/` direktorijuma: `npm run dev`.
5. Otvoriti frontend u pretraživaču

Za prvo, inicijalno podešavanje (pre nego što worker slika postoji u klasteru), potrebno je dodatno: izgraditi worker Docker sliku (`docker build -t batch-worker:latest worker/`) i učitati je u minikube (`minikube image load batch-

worker:latest`). Ovaj korak se ne ponavlja pri svakom narednom pokretanju sistema, sve dok se sadržaj worker slike ne promeni.

4.3.5.Primeri za proveru izvršenog koda

Ispravnost implementacije proverava se na dva nivoa, koji se međusobno dopunjuju:

- Automatizovani test paket (`cd backend && python -m pytest`), koji ne zahteva pokrenut klaster (Kubernetes je mokovan/simuliran u testovima) i proverava logiku particionisanja, agregacije, validacije, orkestracione logike i watch mehanizma u izolaciji. Detaljan pregled test paketa, uključujući broj testova po fajlu i šta svaki fajl pokriva, dat je u poglavlju 4 (Tabela 4.1) i Prilogu 7.4.
- End-to-end demonstracioni skript (`python backend/scripts/e2e_demo.py`), koji zahteva pokrenut ceo sistem (backend, frontend nije neophodan, minikube klaster i mount most) i izvršava crno-kutijasto (black-box) testiranje preko stvarnog HTTP API-ja, uključujući i scenarije otkaza (npr. brisanje chunk fajla usred izvršavanja, radi provere da sistem korektno prelazi u stanje Failed umesto da ostane zaglavljen).

Kao minimalan primer provere: otpremanje CSV datoteke od nekoliko desetina redova sa dva worker-a treba da rezultuje posla koji prolazi kroz stanja Pending → Running → Completed u roku od nekoliko sekundi, uz izlaznu datoteku (output.csv) koja sadrži isti broj redova podataka kao ulazna datoteka (uvećan za jedno zaglavlje), sa dodatom kolonom koja označava da je red obrađen i identifikatorom worker-a koji ga je obradio.

5. Pregled dobijenih rezultata

Ovo poglavlje prikazuje sprovedeno testiranje predloženog rešenja u dva nezavisna aspekta: funkcionalno testiranje (kojim se proverava da sistem ispravno realizuju definisane funkcionalnosti) i testiranje performansi pri promeni broja worker-a (kojim se proverava početna pretpostavka o ponašanju sistema pod paralelizacijom i ispituje da li je sistem ograničen procesorskom snagom ili ulazno-izlaznim operacijama)

5.1. Sistematsko manuelno testiranje graničnih slučajeva

Pre formalizacije testova u automatizovani paket, sproveden je sistematski manuelni prolazak kroz granične slučajeve nad živim sistemom (backend, frontend i stvaran minikube klaster). Ovim prolaskom pronađeno je i otklonjeno pet konkretnih grešaka, prikazanih u Tabeli 4.1, koje predstavljaju stvarne nalaze dobijene tokom razvoja, a ne pretpostavljene ili hipotetičke scenarije.

Tabela 4.1

Redni broj	Opis problema	Rešenje
1	Posao čiji pod-ovi nikada ne dostignu terminalnu fazu (npr. neispravna slika, pod se ne može rasporediti) ostajao je zauvek u stanju Running	Uveden activeDeadlineSeconds
2	Dvostruko slanje POST /run zahteva za isti posao rušilo je gubitnika (proces koji izgubi trku) neobrađenim 500 odgovorom, usled Kubernetes 409 greške pri pokušaju kreiranja Job-a sa imenom koje već postoji	Uveden eksplicitan, čist 409 odgovor
3	UTF-8 BOM oznaka na početku otpremljene CSV datoteke prljala naziv prve kolone u zaglavlju	Uvedeno utf-8-sig dekodiranje
4	Neuskladjeni redovi CSV datoteke proizvodili su tiho pogrešno poravnat izlaz	Uvedeno odbijanje takvih datoteka već pri otpremanju, sa preciznom porukom greške, uz ponovnu validaciju i u modulu za particionisanje

Pronađene i otklonjene greške

5.2. Automatizovani test paket

Manuelno pronađeni scenariji formalizovani su u automatizovani test paket (`backend/tests/ `, pokreće se preko `pytest`), koji u trenutku pisanja rada ima 82 testa raspoređena u sedam fajlova, prikazanih u Tabeli 4.2. Kubernetes je u celom paketu mokovan (simuliran preko `unittest.mock`), tako da paket ne zahteva pokrenut klaster i može se izvršiti u svakom razvojnom okruženju.

Tabela 4.2

Fajl	Broj testova	Pokriva
test_job_service.py	20	Prelaze stanja run_job-a i trke pri istovremenom pokretanju; reconcile_status (pojedinačni posao: preskakanje ne-Running poslova, no-op kad Kubernetes Job ne postoji, ponašanje kad je klaster nedostupan, prelazi u Failed/ Completed uz čišćenje); reconcile_jobs_batch (grupni put: preskače Kubernetes poziv kad nema Running poslova, jedan poziv za više Running poslova, uparivanje/prelaz po id-u posla, nepromenjeno stanje za neuparene/nedostupne slučajeve); get_job_workers; cleanup_job
test_watcher.py	17	Parsiranje id-a posla iz oznake Kubernetes objekta; obradu događaja (uključujući test od događaja do stanja Completed bez mokovanja rekoncilijacione funkcije); direktno pozivanje reconcile_from_k8s_job (nepoznat id, preskakanje ne-Running poslova, prelaz u Failed, izostanak dvostruke primene pri trci sa već terminalnim poslom); tok događaja (raspoređivanje, rano zaustavljanje, otpornost na jedan loš događaj, povratak na grešku konekcije); ponovno povezivanje watcher-a (tajming/odlaganje)
test_partitioner.py	14	Raspodelu ostatka pri neravnomernoj podeli; worker_count jednak broju redova; jedan worker; ponavljanje zaglavlja u svakom chunk-u; prazna/samo-zaglavlje datoteka; granice worker_count-a; grešku pri nepravilnim redovima sa naznakom broja reda; uklanjanje BOM oznake; citirane zapete/prelome reda; podudaranje rezultata kad se prosledi eksplicitan broj redova
test_aggregator.py	5	Spajanje sa jednim zaglavljem preko više fajlova; grešku pri neusaglašenom zaglavlju; grešku pri nedostajućem fajlu; grešku pri praznom fajlu rezultata
test_job_builder.py	4	Rast proporcionalnog roka sa brojem redova po worker-u; opadanje roka sa porastom worker_count-a za fiksan broj redova; da rok nikad ne padne ispod bazne vrednosti; da se izračunata vrednost stvarno ugrađuje u manifest
test_upload_validation .py	20	Granične slučajeve POST /api/jobs preko test klijenta: proveru produžetka, prazan/samo-zaglavlje/nepravilan CSV, granice worker_count-a, neispravna multipart polja i sl.

Fajl	Broj testova	Pokriva
test_db_schema_sync.py	2	Samoisceljujuću migraciju šeme (dodavanje nedostajućih kolona zastareloj tabeli, i izostanak promene kad se tabela već poklapa sa modelom)

Pregled automatizovanog test paketa

Svi testovi su, u trenutku poslednjeg izvršavanja, prolazili bez greške (82/82). Odvojeno od pytest paketa, end-to-end demonstracioni skript (`backend/scripts/e2e_demo.py`) izvršen je poslednji put sa rezultatom 33/33 uspešnih scenarija.

Rezultati funkcionalnog testiranja potvrđuju početnu pretpostavku da sistem pouzdano realizuje ceo životni ciklus posla: particionisanje, orkestraciju, ažuriranje statusa poslova, agregaciju i čišćenje, uključujući i ponašanje pri istovremenom pristupu (konkurentni poslovi, dvostruko slanje zahteva) i pri otkazima (zaglavljeni pod-ovi, nestanak fajla usred izvršavanja).

5.3. Testiranje performansi pri promeni broja worker-a

Za proveru pretpostavke o ponašanju sistema pod paralelizacijom razvijen je poseban benchmark mehanizam, nezavisan od funkcionalnog test paketa, sastavljen od tri dela:

1. Instrumentacija vremena na nivou pojedinačnog pod-a - svaki worker meri (preko `time.monotonic()`) vreme čitanja svog chunk-a, vreme izvršavanja transformacije i vreme upisa rezultata, i upisuje ove vrednosti u `timing_{index:03d}.json` fajl, koji čišćenje posla ne dira, pa preživljava i posle završetka posla. Ovo je najdirektniji dokaz o raspodeli vremena unutar jednog pod-a, jer meri stvarnu podelu
2. Model simulirane cene izračunavanja - `sleep(compute_fixed_s + compute_per_row_s × broj_redova_u_chunk-u)`, sa podrazumevanim vrednostima `compute_fixed_s = 0,3 s` i `compute_per_row_s = 0,000001 s` (jedan mikrosekund po redu)
3. Poseban „noop-transform" režim rada worker-a (upravljan promenljivom okruženja NOOP_TRANSFORM), u kom worker u potpunosti preskače korak izračunavanja i samo prepisuje chunk nazad kao rezultat bez izmene. Poređenjem vremena izvršavanja u ovom režimu sa vremenom izvršavanja u režimu sa izračunavanjem (baseline), izoluje se doprinos ulazno-izlaznih operacija

Namenska skripta (`backend/scripts/worker_sweep.py`) automatizuje merenje: za svaku ulaznu datoteku i svaku vrednost broja worker-a iz unapred definisanog skupa `[1, 2, 4, 8, 10, 12, 14, 16, 20, 24, 32]`, otprema datoteku, pokreće posao, prati status (interval osmatranja 1 sekunda, vremensko ograničenje 180 sekundi) do terminalnog stanja, meri ukupno proteklo vreme od predaje do terminalnog stanja (wall_s), i učitava nazad svaki pojedinačan JSON fajl radi izračunavanja proseka/minimuma/maksimuma vremena čitanja, izračunavanja i pisanja po worker-ima. Rezultati se upisuju u CSV fajl po ulaznoj datoteci u `backend/scripts/sweep_results/`.

Iz izmerenog wall_s izvode se tri dodatne metrike, prikazane u tabelama 4.3 i 4.4: ubrzanje (speedup = wall_s pri jednom worker-u / wall_s pri N worker-a), paralelna efikasnost (efficiency = speedup / N) i propusnost (throughput = broj_redova / wall_s).

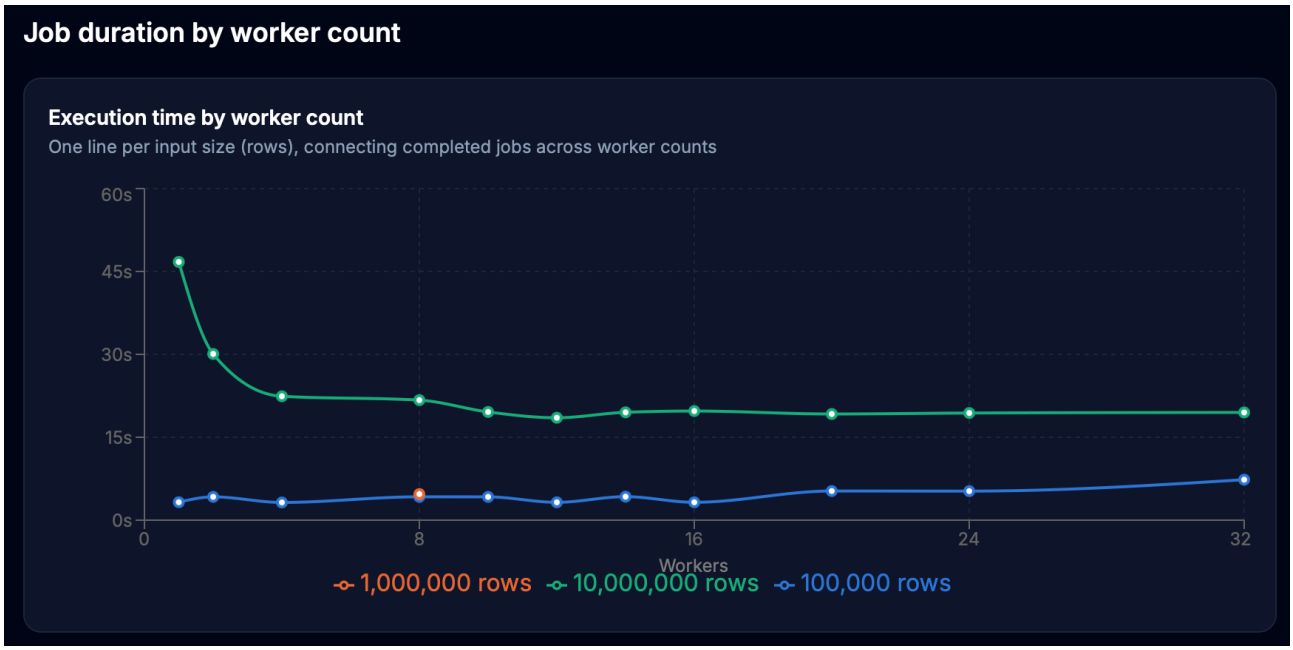
5.3.1. Rezultati za skup podataka od 10.000.000 redova

Tabela 4.3 prikazuje izmerene i izvedene metrike za veliki skup podataka. Vrednosti wall_s su zaokružene radi preglednosti; tačne vrednosti nalaze se u odgovarajućem CSV fajlu rezultata sweep-a.

Tabela 4.3

Broj workera	wall_s [s]	ubrzanje	efikasnost	propusnost [redova/s]
1	54,0	1,00	1,00	~125.000
2	35,8	1,51	0,76	~279.000
4	29,0	1,86	0,47	~345.000
8	25,6	2,11	0,26	~391.000
10	24,6	2,20	0,22	~407.000
12	24,5	2,20	0,18	~408.000
14	25,2	2,14	0,15	~397.000
16	25,5	2,12	0,13	~392.000
20	25,1	2,15	0,11	~398.000
24	25,3	2,13	0,09	~395.000
32	25,6	2,11	0,07	~391.000

Rezultati merenja, 10.000.000 redova, bazni (baseline) režim



Slika 3: Vreme izvršavanja posla u zavisnosti od broja worker-a



Slika 4: Pregled metrika dobijenih benchmark sweep-om

Iz Tabele 4.3 i Slika 3–4 vidljiv je jasan obrazac: ubrzanje raste strmo od 1 do oko 8 worker-a, dostiže maksimum od približno 2,2 puta oko 10–12 worker-a, a zatim ulazi u plato, te dodatno povećavanje broja worker-a (sa 8 na 32) ne donosi merljivo poboljšanje, već performanse ostaju u uskom opsegu (wall_s između otprilike 24,5 i 25,6 sekunde) sve do 32 worker-a. Paralelna efikasnost, u skladu sa tim, opada sa porastom broja worker-a. Sa idealnih 1,00 na samo 0,07 pri 32 worker-a, što znači da je svaki dodatni worker iznad plato tačke sve manje isplativ u smislu doprinosa ubrzanju.

Ovakav obrazac je u skladu sa Amdahl-ovim zakonom (ubrzanje je ograničeno delom posla koji se ne može paralelizovati), ali njegov konkretan uzrok u ovom sistemu je preciznije objašnjen instrumentacijom vremena po pod-u: za chunk od, na primer, 10.000.000/8 = 1.250.000 redova, simulirano vreme izračunavanja iznosi samo `0,3 + 1.250.000 × 0,000001 = 1,55` sekundi, što je zanemarljivo malo u odnosu na ukupno vreme izvršavanja od oko 26 sekundi. Dominantan deo vremena, dakle, otpada na čitanje i pisanje preko deljenog hostPath/`minikube mount` skladišta, a ne na izračunavanje, što direktno potvrđuje početnu pretpostavku da će deljeno skladište, a ne procesorska snaga, predstavljati usko grlo sistema. Pošto svi pod-ovi dele isti fizički 9p most nezavisno od toga koliko ih je pokrenuto istovremeno, propusnost tog mosta postaje gornja granica koju veći broj worker-a ne može prevazići. Što objašnjava zašto propusnost sistema (poslednja kolona Tabele 4.3) staje na platou od otprilike 390.000 - 410.000 redova u sekundi nezavisno od toga da li je angažovano 8, 16 ili 32 worker-a.

Za razliku od wall_s, ubrzanja i efikasnosti (koje su izmerene, empirijske veličine), vreme simuliranog izračunavanja po pod-u je egzaktno izvodljivo iz poznate formule i poznate veličine chunk-a, pošto su obe komponente formule (compute_fixed_s i compute_per_row_s) fiksne, dokumentovane konstante sistema, a veličina chunk-a je po dizajnu jednaka broj_redova / broj_worker-a. Tabela 4.3a prikazuje ovu egzaktnu vrednost uporedo sa izmerenim wall_s, radi jasnijeg uvida u to koliki udeo ukupnog vremena zaista otpada na (simulirano) izračunavanje.

Tabela 4.3a

Broj worker-a	Redova po workeru	vreme izracunavanja po podu [s]	izmereno wall_s [s]	udeo izracunavanja u wall_s [%]
1	10.000.000	10,3	54,0	~19
2	5.000.000	5,3	35,8	~15
4	2.500.000	2,8	29,0	~10
8	1.250.000	1,55	25,6	~6
16	625.000	0,93	25,5	~4

Broj worker-a	Redova po workeru	vreme izracunavanja po podu [s]	izmereno wall_s [s]	udeo izracunavanja u wall_s [%]
32	312.500	0,61	25,6	~2

Udeo izračunavanja u ukupnom vremenu izvršavanja, 10.000.000 redova

Tabela 4.3a čini eksplicitnim ono što je u već pomenuti: udeo izračunavanja u ukupnom vremenu izvršavanja pada sa otprilike 19% pri jednom worker-u na svega oko 2% pri 32 worker-a. Ovo je i očekivano (sam model predviđa da vreme izračunavanja po pod-u opada gotovo linearno sa porastom broja worker-a, pošto svaki dobija sve manji chunk), ali potvrđuje da, nezavisno od toga koliko se posao usitni, preostalih 80-98% vremena izvršavanja otpada na nešto što izračunavanje ne objašnjava - čitanje ulaza, pisanje izlaza i sinhronizaciju preko deljenog 9p mosta, uz fiksni deo troška same Kubernetes orkestracije (kreiranje pod-a, zakazivanje, pokretanje kontejnera) koji je prisutan bez obzira na veličinu chunk-a.

Ovaj nalaz je moguće uporediti i sa teorijskim predviđanjem Amdahl-ovog zakona: ako se pretpostavi da je izmereni udeo paralelizabilnog dela posla pri jednom worker-u jednak udelu izračunavanja iz Tabele 4.3a (~19%), Amdahl-ov zakon predviđa teorijsku granicu ubrzanja od `1 / (1 − 0,19) ≈ 1,23` puta pri beskonačnom broju worker-a, što je znatno niže od stvarno izmerenog ubrzanja od ~2,2 puta (Tabela 4.3). Ovo prividno odstupanje od teorijskog predviđanja objašnjivo je time što Amdahl-ov zakon u svom klasičnom obliku pretpostavlja da se i serijski deo posla ne skraćuje sa porastom broja worker-a, dok se u ovom sistemu i sam I/O deo posla (čitanje/pisanje) delom skraćuje sa usitnjavanjem chunk-a (manji chunk znači i manje podataka za čitanje/pisanje po pod-u, iako se, usled deljenog mosta, taj deo posla ne skraćuje proporcionalno broju worker-a, već znatno sporije, što upravo i prouzrokuje plato umesto linearnog rasta). Sistem se, drugim rečima, ponaša između dva idealizovana teorijska slučaja: potpuno serijskog I/O dela koji se uopšte ne skraćuje, i potpuno paralelizovanog I/O dela koji bi omogućio linearno skaliranje.

5.3.2.Rezultati za skup podataka od 100.000 redova

Nakon obavljenih eksperimenata sistema koristeći ulaz veličine 100.000 redova dobijaju se sledeći rezultati:

Tabela 4.4

Broj worker-a	wall_s [s]	ubrzanje	efikasnost	propusnost [redova/s]
1	2,0	1,0	1,0	~50.000
2	2,35	0,85	0,43	~42.500

Broj worker-a	wall_s [s]	ubrzanje	efikasnost	propusnost [redova/s]
4	2,90	0,69	0,17	~34.500
8	2,85	0,7	0,09	~35.000
10	2,75	0,73	0,07	~36.400
12	2,15	0,93	0,08	~46.500
14	2,75	0,73	0,05	~36.400
16	2,9	0,69	0,04	~34.500
20	3,6	0,56	0,03	~27.800
24	3,7	0,54	0,02	~27.000
32	4,3	0,47	0,01	~23.300

Rezultati merenja, 100.000 redova, bazni (baseline) režim

Rezultati za mali skup podataka pokazuju suštinski drugačiju, i sa stanovišta naivne pretpostavke „više worker-a je uvek brže”: ubrzanje ostaje ispod 1 za gotovo sve vrednosti broja worker-a (izuzev blagog, lokalnog poboljšanja oko 12 worker-a), a sa 32 worker-a sistem je stvarno sporiji nego sa jednim (wall_s od 4,30 s naspram 2,00 s), odnosno ubrzanje od svega 0,47. Ovo je direktna, empirijski potvrđena demonstracija režima u kome dominira fiksni trošak po poslu (Kubernetes Job/Pod kreiranje i raspoređivanje, pokretanje kontejnera, uspostavljanje veze preko 9p mosta po pod-u). Trošak koji raste približno linearno sa brojem angažovanih pod-ova, dok korisni posao po pod-u (chunk od svega nekoliko hiljada redova) opada, pa dodatni paralelizam ne samo da ne pomaže, već aktivno šteti. Ovo je u potpunosti u skladu sa Amdahl-ovim zakonom, gde je zapaženo da na malim/trivijalnim skupovima podataka veći broj worker-a pogoršava, a ne poboljšava performanse.

Poređenjem tabela 4.3 i 4.4 jasno se vidi i da je propusnost sistema za veliki skup podataka (~390.000–410.000 redova/s na platou) za red veličine veća od propusnosti za mali skup podataka (~23.000–50.000 redova/s), što dodatno potvrđuje da je ograničavajući faktor sistema deljeni, agregatni kapacitet skladišnog mosta, a ne broj angažovanih worker-a samog po sebi. Te da veliki skup podataka ima dovoljno posla da amortizuje fiksni trošak po pod-u i da iskoristi (do granice zasićenja) taj deljeni kapacitet, dok mali skup podataka tu granicu nikad ni ne dosegne, pa je u potpunosti dominiran fiksnim troškovima orkestracije.

Isti egzaktan račun sproveden u Tabeli 4.3a (za veliki skup podataka) može se primeniti i ovde, i upravo taj račun objašnjava zašto je efekat kod malog skupa podataka suprotan efektu kod velikog. Za 100.000 redova podeljenih na, na primer, 8

worker-a, chunk iznosi 12.500 redova, a vreme simuliranog izračunavanja po pod-u je `0,3 + 12.500 × 0,000001 = 0,3125` sekundi, što je već samo po sebi manje od izmerenog wall_s od 2,85 sekundi (Tabela 4.4), ali, za razliku od velikog skupa podataka, ovde fiksna komponenta troška (compute_fixed_s = 0,3 s, koja ne zavisi od veličine chunk-a) čini gotovo ceo taj iznos. Smanjivanjem chunk-a (povećavanjem broja worker-a) vreme izračunavanja po pod-u praktično prestaje da opada, dok fiksni trošak same Kubernetes orkestracije po pod-u (kreiranje, zakazivanje, pokretanje kontejnera, uspostavljanje pristupa mount-u) ostaje približno konstantan po pod-u i, pomnožen većim brojem pod-ova, počinje da dominira ukupnim vremenom. Što je upravo mehanizam koji objašnjava zašto wall_s u Tabeli 4.4 posle izvesne tačke počinje da raste sa brojem worker-a, umesto da opada ili stagnira kao kod velikog skupa podataka. Drugim rečima: kod velikog skupa podataka, dodavanje worker-a smanjuje chunk dovoljno da se uštedi vreme na I/O delu, i ta ušteda nadmašuje fiksni trošak po pod-u sve dok se ne dosegne zasićenje deljenog mosta; kod malog skupa podataka, chunk je već od starta dovoljno mali da dalje smanjivanje donosi zanemarljivu uštedu, pa fiksni trošak po pod-u, pomnožen sve većim brojem pod-ova, postaje gubitak, a ne dobitak.

5.3.3.Diskusija i provera početnih pretpostavki

Rezultati testiranja performansi u celini potvrđuju obe povezane pretpostavke:

- da jednočvorni minikube klaster, uprkos ograničenjima skladišta, dovoljno dobro demonstrira i omogućava merenje modela orkestracije,
- da je usko grlo sistema, u trenutnoj konfiguraciji sa hostPath/`minikube mount` skladištem, ulazno-izlazni kanal, a ne procesorska snaga

Ova druga pretpostavka je potvrđena na dva nezavisna načina: direktno, kroz instrumentaciju vremena po pod-u koja pokazuje da simulirano vreme izračunavanja čini zanemarljiv deo ukupnog vremena i posredno, kroz sam graf ubrzanja.

Vredi napomenuti i granicu pouzdanosti ovih rezultata: benchmark skripta u opisanoj konfiguraciji izvršava tačno jedno merenje po kombinaciji (režim, broj_redova, broj_worker-a), bez ugrađenog ponavljanja na nivou same skripte (za razliku od grafikona vremena izvršavanja u Jobs panelu (Slika 3) koji prikazuje prosek više stvarnih, organski prikupljenih pokretanja pri istom broju worker-a i koji nezavisno potvrđuje isti oblik krive). Rezultati u tabelama 4.3 i 4.4 stoga treba čitati kao jednu reprezentativnu, a ne statistički uprosečenu meru za svaku tačku.

Dodatna, kvalitativna provera sprovedena je i ručnim praćenjem opterećenja sistema tokom tačke sa 24 worker-a (najzahtevnija tačka u skupu od 10 miliona redova koja se ručno posmatrala): na host mašini praćen je proces `minikube mount` (preko `pgrep`/`top`), a unutar minikube virtuelne mašine iskorišćenost CPU-a naspram vremena čekanja na I/O (preko `vmstat`). Ovo posmatranje je u skladu sa nalazom iz tabela: CPU jezgra nisu bila zasićena, dok je udeo vremena provedenog u čekanju na

I/O bio primetan, što dodatno potvrdjuje zaključak da sistem, u ovoj konfiguraciji, čeka na skladište, a ne na procesor.

Vredi razgraničiti i dve komponente ukupnog vremena koje se u samom sistemu prate zasebno na nivou pojedinačnog posla (schemas/job.py, Prilog 7.4): vreme čekanja (queue_time_s, razlika između trenutka kreiranja posla i trenutka kada je stvarno prešao u Running) i vreme izvršavanja (execution_time_s, razlika između trenutka prelaska u Running i trenutka dostizanja terminalnog stanja). Benchmark sweep skripta meri isključivo drugu veličinu, wall_s koja se računa od trenutka predaje posla preko API-ja do terminalnog stanja, dakle obuhvata i particionisanje i predaju Job-a Kubernetesu, pre nego što bilo koji pod počne da radi. U benchmark uslovima (sekvencijalno izvršavanje jednog posla u trenutku, bez konkurencije za resurse), vreme čekanja je zanemarljivo malo i praktično se gubi u šumu merenja. Ipak, ova razlika postaje bitna izvan benchmark konteksta: u organskom, svakodnevnom korišćenju sistema, gde više poslova različitih korisnika mogu istovremeno čekati na ograničen broj raspoloživih CPU jezgara klastera, queue_time_s bi počeo da raste nezavisno od execution_time_s i taj rast bi predstavljao drugačiji, ranije u sistemu smešten oblik uskog grla, razlika koju bi buduće inkrementalne ispravke trebalo posebno da mere ako bi se sistem testirao pod istovremenim opterećenjem više poslova, a ne kao u ovom radu, jednim poslom u datom trenutku.

5.3.4.Pretnje validnosti benchmark eksperimenta

Pored granice pouzdanosti navedene u prethodnom odeljku, vredi eksplicitno navesti i druge faktore koji mogu uticati na tumačenje rezultata: merenja su sprovedena na jednoj razvojnoj mašini koja tokom izvršavanja skripte nije bila izolovana od drugih procesa (operativni sistem, pozadinski servisi), pa kolebanja u pozadinskom opterećenju mogu unositi šum u pojedinačna merenja, posebno pri manjim vremenima izvršavanja (skup od 100.000 redova, gde su apsolutne razlike u sekundama male, pa je relativni uticaj šuma veći); minikube klaster deli fizičke resurse sa domaćinom operativnim sistemom, pa opterecenje procesora ili diska od strane drugih aplikacija na istoj mašini nisu bili posebno kontrolisani; i, konačno, sama benchmark skripta izvršava poslove sekvencijalno, jedan po jedan, pa mereni wall_s za veći broj worker-a ne uključuje efekat istovremenog izvršavanja više poslova različitih korisnika, koji bi u realnom, višekorisničkom razmeštanju dodatno opteretio deljeni I/O kanal identifikovan kao usko grlo sistema.

5.3.5.Veza sa uporednom odradjenom analizom

Rezultati ovog poglavlja imaju i značaj koji nadilazi samu proveru pretpostavki o ponašanju sistema - oni potkrepljuju i argument da direktna upotreba Kubernetes primitiva omogućava vrstu transparentnosti, koju rešenja poput Spark-a ili AWS Batch-a, po prirodi svoje arhitekture otežavaju: merenje vremena po pod-u, koja je omogućila da se udeo izračunavanja u ukupnom vremenu izvršavanja tačno izračuna, a ne proceni, bila je moguća upravo zato što je svaki worker obična, direktno

kontrolisana Kubernetes Pod instanca čiji se kod (worker/main.py) u potpunosti nalazi u domenu ovog rada. Kod sistema poput Spark-a, ekvivalentno merenje bi zahtevalo zalaženje u Spark-ov sopstveni sloj metrika (Spark UI, event log), koji meri izvršavanje na nivou Spark zadataka i faza, a ne na nivou Kubernetes pod-a, dok bi kod upravljanih cloud servisa (AWS Batch) ovakva instrumentacija na nivou pojedinačnog izvršioca po pravilu uopšte ne bila dostupna korisniku. Ova mogućnost preciznog merenja predstavlja praktičnu potvrdu jedne od ciljnih karakteristika definisanih (merljivost i observabilnost), a istovremeno objašnjava zašto je upravo predloženo rešenje omogućilo da se u ovom radu, pored funkcionalne ispravnosti, empirijski dokaže i konkretan, kvantifikovan uzrok ponašanja sistema pod paralelizacijom.

6. Zakljucak

Zadatak diplomskog rada definisao je osam konkretnih funkcionalnosti koje platforma treba da poseduje:

- predaju posla sa otpremanjem CSV datoteke
- konfigurisanje broja worker pod-ova
- particionisanje skupa podataka po redovima između worker-a
- automatsko kreiranje i upravljanje Kubernetes Job i Pod resursima
- distribuirano izvršavanje jednostavnog zadatka obrade preko worker pod-ova
- praćenje statusa izvršavanja (Pending, Running, Completed, Failed)
- agregaciju parcijalnih rezultata u konačan izlaz
- automatsko čišćenje Job-ova, Pod-ova i privremenih datoteka nakon završetka.

Sve navedene funkcionalnosti su implementirane u potpunosti, i sve su pokrivene automatizovanim testovima i/ili end-to-end demonstracionim skriptom. Sistem je, dakle, u celini ispunio funkcionalni obim definisan u zadatku rada.

Van striktno funkcionalnog obima, rad je ostvario i eksplicitno postavljen edukativni cilj - da dokaže sposobnost Kubernetesa da pouzdano orkestrira distribuirano izvršavanje zadataka - kroz konkretnu demonstraciju upotrebe kontejnerizacije (Docker), Kubernetes orkestracije (Job, Pod, Indexed completion mode), distribuiranog izvršavanja, konkurentnih worker-a i izvršavanja bez tragova (stateless) pojedinačnog worker-a. Pored toga, rad je otišao korak dalje od pukog dokazivanja funkcionalnosti i empirijski izmerio ponašanje sistema pod paralelizacijom, čime je ostvaren i dodatni, neformalno postavljen, ali suštinski važan cilj da tvrdnje o performansama sistema budu podržane merenjem, a ne pretpostavkom.

Najznačajniji rezultat rada je funkcionalan sistem, koji korisniku omogućava da, bez poznavanja Kubernetes API-ja ili kubectl komandi, otpremi podatke, pokrene distribuiranu obradu i preuzme rezultat, uz punu transparentnost procesa kroz status i metrike u realnom vremenu. Posebno vredni rezultati, koji nadilaze minimalni obim zadatka, jesu: dvostruki (watch + poll) mehanizam rekoncilijacije statusa, koji čini sistem otpornim na propuštene događaje bez gubljenja reagovanja u realnom vremenu; sistematsko, dvoslojno testiranje (82 automatizovana testa + end-to-end demonstracija nad stvarnim klasterom), koje je tokom razvoja stvarno pronašlo i otklonilo pet konkretnih grešaka; i, najznačajnije, rigorozno sprovedena benchmark analiza koja empirijski potvrđuje da je sistem ograničen deljenim I/O kanalom, uz kvantifikovano ubrzanje (~2,2×) i jasno identifikovanu tačku zasićenja (plato od 8 do 32 worker-a za veliki skup podataka).

Rad takođe ima jasno definisana ograničenja, koja su, u skladu sa principom poštene i kritičke samoprocene, ovde eksplicitno navedena:

- Održavan poseban proces radi potrebe `minikube mount`, nema nadzor ili automatski restart, i može ući u „stale” stanje (proces živ, ali 9p sesija mrtva) koje se ne detektuje prostom proverom da li proces radi
- Rok izvršavanja posla (activeDeadlineSeconds), iako proporcionalan količini posla po worker-u, i dalje je fiksna formula, a ne vrednost izvedena iz stvarno izmerene propusnosti sistema
- Validacija sadržaja CSV datoteke - sistem ne proverava semantičku ispravnost podataka

- Odsustvo autentifikacije i podrška za samo jednog pisca baze podataka (SQLite) - isključuje višekorisnički rad ili više istovremenih instanci backend-a bez dodatnih izmena
- Odsustvo ponovnih pokušaja (`backoffLimit = 0`) - jedan prolazni otkaz pod-a (npr. privremena greška pri otpremanju fajla) obara ceo posao, umesto da se ponovi samo taj deo posla
- Testiranje ograničeno na jednočvorni klaster - nema stvarnog višečvornog klastera niti cloud okruženja, a skladište je lokalni hostPath, a ne mrežno skladište, što ograničava zaključke o ponašanju sistema van ovog konteksta
- Simulirano, a ne stvarno izračunavanje u worker-u - namerna odluka koja omogućava kontrolisano merenje, ali znači da rezultati o odnosu I/O i računanja važe za profil opterećenja korišćen u eksperimentu, a ne nužno za svaki stvaran zadatak obrade
- Odsustvo automatizovanog testiranja frontend-a - React aplikacija je proverena samo kroz uspešan build (`vite build`) i statičku analizu (`oxlint`), bez automatizovanih testova ponašanja u pretraživaču.

Uprkos navedenim ograničenjima, koja su, treba naglasiti, uglavnom posledica svesno suženog obima rada (a ne slučajnog previda), predloženo rešenje ima neposrednu primenljivost kao obrazovni alat za demonstraciju Kubernetes Job/Indexed completion mode obrasca, kao osnova za brzo prototipisanje distribuiranih paketnih pipeline-ova u lokalnom razvojnom okruženju, pre prelaska na produkcionu infrastrukturu; i/ili kao arhitekturni temelj za dalji razvoj u pravcu opšte platforme za distribuiranu obradu podataka.

Na osnovu identifikovanih ograničenja mogući pravci daljeg rada:
- Zamena hostPath/`minikube mount` skladišta objektnim skladištem (npr. AWS S3), radi uklanjanja 9p zavisnosti i omogućavanja rada na stvarnom višečvornom klasteru.
- Izvođenje konstante u formuli za rok izvršavanja iz stvarno izmerene propusnosti sistema (npr. iz baznih benchmark rezultata), umesto iz fiksne, ručno podešene vrednosti.
- Uvodjenje mogućnosti ponovnih pokušaja po pod-u u slučaju otkaza, uz očuvanje determinističke dodele chunk-a preko completion index-a.
- Uvođenje generičkog interfejsa za obradu (validate/partition/execute), koji bi omogućio podršku za dodatne tipove ulaza (TXT, JSON, slike, logovi) bez izmene orkestracionog sloja
- Premeštanje sistema na upravljani Kubernetes servis u oblaku (Google Kubernetes Engine), radi uklanjanja ograničenja lokalne mašine i omogućavanja testiranja na stvarnom višečvornom klasteru sa objektnim skladištem.
- Dodavanje autentifikacije, korisničkog upravljanja CPU/RAM resursima po poslu i podrške za dodatne formate ulaza (ZIP arhive)
- Uvođenje inteligentnog planiranja izvršavanja (prioritet, rok, budžet) i procene potrebnih resursa na osnovu istorijskih podataka o prethodnim poslovima (AI-bazirana procena resursa), što bi direktno nadograđivalo benchmark infrastrukturu razvijenu u ovom radu

7. Literatura

- The Kubernetes Authors, „Jobs, «Indexed» Completion Mode i CronJobs,” https://kubernetes.io/docs/ (odeljci „Workloads” → „Jobs”), pristupano septembar 2026
- minikube zvanična dokumentacija https://minikube.sigs.k8s.io/docs/, pristupano septembar 2026.
- „FastAPI - zvanična dokumentacija,” https://fastapi.tiangolo.com/, pristupano septembar 2026.
- „SQLAlchemy - zvanična dokumentacija,” https://docs.sqlalchemy.org/, pristupano septembar 2026.
- Kubernetes SIGs, „Kubernetes Python Client - zvanična dokumentacija i primeri,” https://github.com/kubernetes-client/python, pristupano septembar 2026.
- Meta Platforms Inc., „React - zvanična dokumentacija,” https://react.dev/, pristupano septembar 2026.
- „Vite — zvanična dokumentacija,” https://vitejs.dev/, pristupano septembar 2026.
- Docker Inc., „Docker - zvanična dokumentacija,” https://docs.docker.com/, pristupano septembar 2026.
- Dean Jeffrey, Ghemawat Sanjay, „MapReduce: Simplified Data Processing on Large Clusters,” OSDI, 2004
- Zaharia Matei i dr., „Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” NSDI, 2012
- Amdahl Gene M., „Validity of the single processor approach to achieving large scale computing capabilities,” AFIPS Conference Proceedings, vol. 30, 1967, str. 483–485.
- Apache Software Foundation, „Apache Airflow - zvanična dokumentacija,” https://airflow.apache.org/docs/, pristupano septembar 2026.
- „Argo Workflows - zvanična dokumentacija,” https://argoproj.github.io/argo-workflows/, pristupano septembar 2026.
- Amazon Web Services, „AWS Batch - zvanična dokumentacija,” https://docs.aws.amazon.com/batch/, pristupano septembar 2026.
- „Celery - zvanična dokumentacija,” https://docs.celeryq.dev/, pristupano septembar 2026.
- Shafranovich Y., „RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files,” oktobar 2005, https://www.rfc-editor.org/rfc/rfc4180, pristupano septembar 2026.

8. Prilozi

Ova sekcija sadrži dopunski materijal koji upotpunjuje opise iz prethodnih poglavlja rada. Obuhvata kompletnu strukturu repozitorijuma, specifikaciju konfiguracije i modela podataka kao i primer ulaza i izlaza CSV formata.

8.1. Kompletna struktura repozitorijuma

```
backend/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   └── db.py
│   ├── models/
│   │   └── job.py
│   ├── schemas/
│   │   ├── job.py
│   │   └── metrics.py
│   ├── api/
│   │   ├── jobs.py
│   │   └── metrics.py
│   ├── services/
│   │   ├── job_service.py
│   │   ├── partitioner.py
│   │   ├── aggregator.py
│   │   └── metrics_service.py
│   └── k8s/
│       ├── client.py
│       ├── job_builder.py
│       ├── orchestrator.py
│       └── watcher.py
├── tests/
└── scripts/
    ├── e2e_demo.py
    ├── worker_sweep.py
    ├── sweep_both_modes.sh
    └── cleanup_jobs.py
worker/
├── main.py
└── Dockerfile
frontend/src/
├── App.jsx
├── api.js
├── components/
│   ├── UploadForm.jsx
```

```
│   ├── JobList.jsx
│   ├── JobDetailModal.jsx
│   ├── StatusBadge.jsx
│   ├── PipelineOverview.jsx
│   ├── JobDurationChart.jsx
│   └── MetricsPanel.jsx
└── utils/
    ├── format.js
    └── chartSeries.js
```

8.2. Kompletna specifikacija konfiguracionih parametara

Svi parametri se nalaze u backend/app/core/config.py i mogu se prilagoditi preko promenljivih okruženja ili .env fajla, bez izmene koda.

Tabela 7.3

Parametar	Podrazumevana vrednost	Značenje
storage_dir	./storage	Koreni direktorijum za fajlove
database_url	sqlite:///./app.db	Konekcioni string ka bazi
max_workers_per_job	32	Operativna gornja granica broja worker-a po poslu
worker_image	batch-worker:latest	Ime Docker slike koju pod-ovi
data_mount_path	/data	Putanja na koju se hostPath volumen montira unutar pod-a
job_deadline_base_seconds	60	Bazna komponenta formule za activeDeadlineSeconds
job_deadline_seconds_per_ro	0,5	Komponenta formule po redu-po-worker-u
kube_namespace	default	Kubernetes namespace u kom se posao izvršava
kube_context	None	Kubernetes kontekst
noop_transform	FALSE	Prekidač za benchmark režim bez izračunavanja
compute_fixed_s	0,3	Fiksna komponenta simulirane cene izračunavanja [s]
compute_per_row_s	1	Komponenta simulirane cene po redu [ns]

Parametar	Podrazumevana vrednost	Značenje
sweep_results_dir	./scripts/sweep_results	Direktorijum za CSV rezultate benchmark sweep-a

Konfigurabilni parametri sistema

8.3. Kompletna specifikacija modela podataka posla

Izvedene veličine (schemas/job.py): queue_time_s = started_at − created_at; execution_time_s = finished_at − started_at; throughput_rows_per_s = row_count / execution_time_s.

Tabela 7.4

Polje	Tip	Napomena
id	string (uuid4 hex)	Primarni identifikator posla
status	enum(Pending/Running/ Completed/Failed)	
worker_count	integer	
row_count	integer	Izračunato jednim prolazom pri otpremanju
original_filename	string	
input_path	string	
result_path	string(nullable)	Popunjava se po zavrsetku agregacije
error_message	string(nullable)	Popunjava se pri prelazu u Failed
worker_snapshot	JSON (nullable)	Snimak statusa pod-ova pre čišćenja
created_at	timestamp	
updated_at	timestamp	
started_at	timestamp	Postavlja se pri prelazu u Running
finished_at	timestamp	Postavlja se pri prelazu u terminalno stanje

Struktura posla u bazi podataka

8.4. Primer strukture ulazne i izlazne CSV datoteke

Primer minimalne ulazne datoteke (input.csv):

Table 7.5

id	vrednost
1	12
2	45
3	8
4	31

Primer ulazne datoteke

Primer odgovarajuće izlazne datoteke nakon obrade sa dva worker-a (output.csv). Svaki red dobija dodatnu kolonu koja beleži da je obrađen i koji je worker (po completion index-u) obradio taj red:

Table 7.6

id	vrednost	processed_by_worker
1	12	0
2	45	0
3	8	1
4	31	1

Primer izlazne datoteke za 2 worker-a